IFN636 Software Life Cycle Management

Assessment 1

Software Requirements Analysis and Design

Event Ticket Booking System

A booking platform that cannot oversell an event

Field	Detail
Full name	Nikhittha Mukkala
Student ID	N12202665
Tutor name	Shinthi Tasnim Himi (Himi)
Tutorial day and time	Wednesday 1:00 pm to 3:00 pm
Project name	Event Ticket Booking System

Project links

Artefact	Link
GitHub repository	https://github.com/nikki505/event-ticket-booking
Jira board	https://connect-team-acmqgqe7.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog
Draw.io diagrams	https://viewer.diagrams.net/?lightbox=1&nav=1#Uhttps%3A%2F%2Fraw.githubusercontent.com%2Fnikki505%2Fevent-ticket-booking%2Fmain%2Fdocs%2Fdiagrams%2Fevent-ticket-booking.drawio
Figma prototype	https://www.figma.com/design/9YK2uHsPeA2ZhLgq0SlIjG/Event-Ticket-Booking-System
EC2 instance ID and name	i-04a1250e9732b2449 / n12202665-nikhittha-eventtix
EC2 public URL	http://32.236.117.199 (Elastic IP, stable)

Note on access. The deployed URL and the Jira board both need an access change before marking. Both are explained in section 5.6 rather than left as a surprise.

1. Problem, Requirements and Project Management

1.1 Problem statement

Small and mid sized event organizers, such as university clubs, community groups and independent promoters, sell tickets through informal channels. Social media posts, spreadsheets and manual bank transfers are common. Three failures recur.

1. Overselling. Capacity is tracked by hand, so an event can accept more bookings than the venue holds. Correcting this afterwards costs refunds and reputation.
2. No single source of truth. The organizer cannot answer the question how many seats are left without reconciling several documents.
3. No self service. Every booking and every cancellation needs the organizer to act.

The Event Ticket Booking System addresses these by holding one authoritative record of events and bookings, by enforcing capacity in software rather than by hand, and by letting attendees book and cancel for themselves.

Why this problem suits the unit.

Most of the system is ordinary data entry. One requirement is not. Capacity must never be exceeded, even when two people try to book the last seat at the same instant. That single constraint gives the design and the test plan something real to reason about instead of being pure create, read, update and delete. The assessment rewards reasoning and verification rather than feature count, so a narrow problem with one genuinely hard constraint is a better fit than a broad problem with none.

1.2 Stakeholders

Stakeholder	Interest	How the design serves it
Event organizer	Publish events, know remaining capacity, see who is coming	Event management, live remaining seat count, attendee list per event
Attendee	Find events, book quickly, cancel without asking anyone	Public browsing, self service booking and cancellation
Venue operator	Accurate headcount for safety and staffing	Capacity enforced by the server, attendee list visible to the organizer
Tutor and marker	Verify traceability and understanding	Requirement IDs carried through backlog, design, prototype and commits

Only the two primary users transact with the system. The venue operator consumes its output and has no login. That is a deliberate scope decision recorded as D002.

1.3 User roles

The brief requires at least two roles. This system defines exactly two. The interface labels them Organiser and Attendee, which is the spelling used throughout the running application and in the screenshots.

Role	Can do	Cannot do
Organizer	Create, update and cancel their own events. View	Book tickets. View or modify another

	bookings and the attendee list for their own events.	organizer event.
Attendee	Browse published upcoming events. Book tickets. View and cancel their own bookings.	Create or modify any event. View another attendee bookings.

Two layers of authorization.

Role alone is not sufficient. The system checks role, meaning is this user an organizer at all, and then ownership, meaning is this actually their event. Both run on the server in the route handler, and ownership is read from the session token rather than from the request body. Hiding a button in the interface is a usability measure, not a security control, because the server assumes the client may be bypassed entirely.

1.4 The two end to end workflows

W1, publish and book. This is the workflow deployed to EC2 and demonstrated.

An organizer registers or signs in, creates an event with a capacity and a price, and the event appears in the public listing. An attendee signs in, opens the event, chooses a quantity and books. A unique booking reference is returned, the remaining seat count drops, and the booking appears in the organizer attendee list.

W2, cancel and release.

An attendee opens My Bookings and cancels. The booking moves to cancelled, the seats return to the event, and the organizer numbers update.

W2 was chosen deliberately. It exercises the same capacity rule as W1 but in the opposite direction, so a fault in capacity accounting has to show up in one of the two. A second workflow touching unrelated data would not have that property.

1.5 Scope

In scope

- Email and password registration and sign in, with a role chosen at registration
- Role and ownership authorization on every protected route
- Event lifecycle: create, list, view, update, cancel
- Booking lifecycle: create, list own, cancel
- Capacity enforced by the server, safe under concurrent requests
- Server side validation returning field level messages
- Storage that survives an application restart
- Manual deployment of W1 to a public EC2 address, with a written procedure

Out of scope, and why

Excluded	Reason
Payment processing	Handling real card data is inappropriate in a student project and would dominate the effort. Events carry a price that is stored and displayed, but no money moves.
Email and SMS notification	Needs an external provider and credentials in the deployment. The booking reference is shown on screen instead.
Seat level selection	Turns a capacity counter into a seat map allocation problem, which is a substantially

	harder design for no extra marks.
Refunds	Follows from payments being out of scope.
Password reset by email	Same external dependency as notifications.
Continuous integration and delivery	Explicitly out of scope per the brief. Deployment is manual and documented.
Multi organizer teams	One event has exactly one owner. Team permissions would need another entity and a full permissions model.

1.6 Assumptions

ID	Assumption	If it proves false
A1	Attendees have an email address and can register themselves	An organizer side booking entry flow would be needed
A2	Events are single session with one date and time	The event model needs a session sub entity
A3	A booking holds 1 to 10 tickets	Add a group booking approval workflow
A4	Capacity is one undifferentiated pool with no ticket tiers	Add a ticket type entity between event and booking
A5	Demonstration load is a handful of users, not a ticket rush	Would need request queueing rather than a conditional update
A6	A single instance is sufficient, no high availability	Would need a load balancer and multiple instances

A4 and A5 are the two most likely to be challenged, so both are answered in the design. A4 by keeping capacity a single number on the event so a tier model could be inserted later without reworking bookings, and A5 by using one atomic conditional update rather than a read followed by a write.

1.7 Measurable success criteria

Each criterion is written so it is objectively pass or fail at demonstration time rather than a matter of opinion.

ID	Criterion	How it is measured	Result
SC1	Both workflows complete on the deployed address	Walk W1 and W2 against the public URL	Pass, section 5.4
SC2	Overbooking is impossible	Capacity 3, attempt to book more than remain	Pass, 409 returned
SC3	Cancellation returns capacity exactly	Book 2 of 3, then cancel	Pass, returned to 3
SC4	Cross role access is refused	Organizer attempts to book, attendee calls organizer route	Pass, 403 both
SC5	Cross ownership access is refused	Organizer A requests organizer B event	Pass, 403

SC6	Invalid input refused with a usable message	Validation table exercised through the interface	Pass, figure 18
SC7	Data survives restart	Restart the process, requery a booking	Pass, section 5.4
SC8	No secret in version control	History scanned, .env ignored	Pass
SC9	Every requirement is traceable	Traceability matrix complete	Pass, section 2.9

1.8 Requirements register

Requirement IDs are the spine of traceability. Every backlog item, design element, prototype frame and commit message references one of these.

Functional requirements

ID	Requirement	Role	Workflow	Priority
R1	A visitor can register with email, password and a chosen role	Both	W1	Must
R2	A registered user can sign in and receive a session token	Both	W1	Must
R3	Protected routes reject requests with no valid session	Both	W1, W2	Must
R4	Organizer only routes reject attendee sessions	Organizer	W1	Must
R5	An organizer can create an event with title, venue, date, capacity and price	Organizer	W1	Must
R6	An organizer can list their own events with remaining seats	Organizer	W1	Must
R7	An organizer can update an event they own	Organizer	W1	Should
R8	An organizer can cancel an event they own	Organizer		Should
R9	Any visitor can browse published upcoming events	Attendee	W1	Must
R10	An attendee can book 1 to 10 tickets and receives a unique reference	Attendee	W1	Must
R11	The system rejects any booking that exceeds remaining capacity	Attendee	W1	Must
R12	An attendee can list their own bookings	Attendee	W2	Must
R13	An attendee can cancel their own booking, releasing the seats	Attendee	W2	Must
R14	An organizer can view the booking list for an event they own	Organizer	W2	Should

Non functional requirements

ID	Requirement	Verification
N1	All input validated on the server, which stays authoritative even if the client is bypassed	Call the API directly with invalid payloads
N2	Passwords stored as salted hashes, never plain text	Inspect a stored user record
N3	No credential, key or token committed to the repository	History scanned, .env ignored

N4	Data persists across application restarts	SC7
N5	W1 reachable at a public address during marking	Open the URL from outside
N6	Inbound access limited to the ports actually required	Review the security group

1.9 Validation rules

Consolidated here because acceptance criteria throughout the backlog refer to them. Every rule is enforced on the server. The client mirrors them only to give faster feedback.

Field	Rule	Message
email	Required, valid format, unique	Enter a valid email address / That email is already registered
password	Required, at least 8 characters	Password must be at least 8 characters
role	Required, organizer or attendee	Select a role
title	Required, 3 to 120 characters	Title must be between 3 and 120 characters
venue	Required, 3 to 160 characters	Venue must be between 3 and 160 characters
startsAt	Required, valid date, in the future	Event date must be in the future
capacity	Required, whole number, 1 to 10000	Capacity must be a whole number of at least 1
price	Required, zero or more, 2 decimal places	Price cannot be negative
quantity	Required, whole number, 1 to 10	You can book between 1 and 10 tickets
quantity vs capacity	Must not exceed remaining seats	Only N seats remain for this event

The last rule is the only one that cannot be checked from the request alone. It depends on the current state of the database and that state can change between checking and saving, so it is enforced inside the atomic update rather than in the validation layer. Section 2.6 develops this.

1.10 Product backlog

Five epics, sixteen user stories, sixty two story points. Estimates use a modified Fibonacci scale sized against a reference story, US04 create an event, which is three points. Points were used rather than hours because the productivity of a developer new to the stack is unknown at the start, while relative sizing stays valid even when the absolute rate turns out to be wrong.

Epic	Title	Requirements	Points	Jira
E1	Accounts and access control	R1, R2, R3, R4, N2	13	SCRUM 1
E2	Event management	R5, R6, R7, R8	12	SCRUM 2
E3	Discovery and booking	R9, R10, R11	16	SCRUM 3
E4	Booking management	R12, R13, R14	11	SCRUM 4
E5	Deployment and documentation	N3, N5, N6	10	SCRUM 5

Story summary

Story	Title	Req	Pts	Iteration
US01	Register with a role	R1, N2	5	1
US02	Sign in and stay signed in	R2, R3	5	1
US03	Role based route protection	R4	3	1
US04	Create an event (reference story)	R5	3	1
US05	View my events	R6	3	1
US06	Update an event	R7	3	1
US07	Cancel an event	R8	3	1
US08	Browse events	R9	3	2
US09	Book tickets	R10	5	2
US10	Capacity is never exceeded	R11	8	2
US11	View my bookings	R12	3	2
US12	Cancel my booking	R13	5	2
US13	View attendees for my event	R14	3	2
US14	Configuration through environment variables	N3	2	1
US15	Deploy W1 to EC2	N5, N6	5	2
US16	README and release tag	N3, N5	3	2

Example of a full story, US10, the only story sized at eight points

As the system, I want to reject bookings beyond remaining capacity, so that the venue is never oversold.

Acceptance criteria

- AC1. Given 2 seats remain, when I attempt to book 3, the response is 409 and nothing is created.
- AC2. Given 2 seats remain, when I book exactly 2, it succeeds and remaining becomes 0.
- AC3. Given 0 seats remain, when I attempt to book 1, the response is 409.
- AC4. Given two requests each booking the last seat arrive at the same moment, exactly one succeeds and the other receives 409. Remaining never goes below zero.
- AC5. Given a rejected booking, no partial or orphaned record exists.

The estimate of eight points reflects risk rather than volume. The naive implementation passes AC1 to AC3 and fails AC4, and that failure is invisible in manual testing. AC4 is what forces a single conditional update, and it is why this story carries the highest estimate in the backlog.

1.11 Iteration plan

	Iteration 1	Iteration 2

Theme	Foundations	Value and deployment
Goal	An organizer can register, sign in and create an event that survives a restart	W1 and W2 both complete against the public address
Committed points	27	35
Stories	US14, US01, US02, US03, US04, US05, US06, US07	US08, US09, US10, US11, US12, US13, US15, US16

Iteration 2 carries more points deliberately. Iteration 1 includes project setup overhead that is not represented in story points, so its effective load is higher than 27 suggests.

Dependencies and critical path

```
US14 -> US01 -> US02 -> US03 -> US04 -> US05 -> US06
                                     -> US07

US04 -> US08 -> US09 -> US10 -> US15 -> US16
                US09 -> US11 -> US12
                US09 -> US13
```

The critical path runs US08, US09, US10, US15, US16. US10 sits on the critical path and is also the highest risk story, which is why it was scheduled early in iteration 2 rather than left until the end.

Ownership

This is an individual assessment, so every item is owned by the student. The ownership column is retained on the Jira board because it makes the single owner constraint explicit rather than implied.

Risk register

ID	Risk	Likelihood	Impact	Mitigation
RSK01	The overbooking guard passes manual testing but fails under concurrency	High	High	Use a conditional atomic update from the start. Write the two simultaneous request test before calling the story done.
RSK02	Deployment consumes far more time than estimated	High	High	Prove the infrastructure early rather than at the end
RSK03	Developer new to the stack, so estimates are unreliable	High	Medium	Relative points not hours, replan iteration 2 using real velocity
RSK04	Iteration 1 dependency chain is linear, one blocked item stalls everything	Medium	Medium	US06 and US07 are the designated items to drop, they are Should not Must
RSK05	A secret is committed and history cannot be rewritten	Low	High	gitignore in the very first commit, before any code
RSK06	The stored seat counter drifts from the real booking total	Medium	Medium	Attendee list shows both numbers side by side, so drift is visible
RSK07	The public address changes before marking	Medium	High	Verify the URL the day before, record the instance ID in the report

1.12 What actually happened against this plan

The plan above describes two time boxed iterations of a week each. That is not how the work actually ran, and saying otherwise would be easy to disprove by comparing the commit history with the sprint dates on the board.

In practice the whole build was compressed into 21 and 22 August 2026. Thirty one commits carry those two dates. The ordering held: configuration and the ignore file first, then accounts and access control, then event management, then booking and the capacity rule, then deployment, then the report. The dependency chain in the plan was followed, but the calendar boxes were not.

Planned	Actual
Iteration 1, one week, 27 points	21 to 22 August, foundations and event management
Iteration 2, one week, 35 points	22 August, booking, cancellation, deployment and report
Review at the end of each iteration	One review, written as the self review on pull request 1

What this costs and what it is worth saying anyway.

Compression removed the main benefit of iterating, which is using measured velocity from the first box to re plan the second. The estimates in this backlog were therefore never corrected against reality, so they remain guesses rather than calibrated numbers. The risk register anticipated exactly this under RSK03 and the mitigation, replanning with real velocity, was the one that could not be applied.

The dependency analysis and the risk register still earned their place. Scheduling US10 early because it sat on the critical path and carried the highest risk was a decision made from the plan, and it was the right one, because that story did take the longest and did need the most rework.

Realized risks

Two risks on this register actually occurred during the project. RSK01 was avoided rather than realized, because the atomic update was written from the start and the concurrency test proved it. RSK02 was partly realized. The deployment did consume more time than estimated, and the cause was not the code but an account restriction described in section 5.3.

1.12 Decision and change log

ID	Decision	Alternatives considered	Why this one
D001	Build an event ticket booking system	IT support tickets, fitness class booking, equipment rental	All four have two roles and two workflows. Only ticket booking has a hard capacity rule that must hold under concurrency.
D002	Exactly two roles	A third read only venue role, a platform admin role	Two roles keep the authorization matrix small enough to test exhaustively.
D003	Keep price, exclude payments	Payment sandbox, or remove price entirely	Keeping price preserves realism and one more validation rule at no integration cost.
D004	Store the remaining seat count rather than computing it	Compute from bookings on every read	A stored number can be changed with one atomic conditional update. A computed one forces a read then write with a race window

			between.
D005	Cancellation is a status change, never a delete	Delete the row	Deleting an event orphans every booking that references it, and deleting a booking makes the seat arithmetic impossible to audit.
D006	Identical message for unknown email and wrong password	Distinct, friendlier messages	Distinct messages let anyone discover which addresses are registered.
D007	Authorization on the server, interface hiding is cosmetic	Rely on conditional rendering	Anything enforced only in the browser can be bypassed by calling the API directly.
D008	Requirement IDs used verbatim across every artefact	Assemble the matrix at the end by inspection	Deciding IDs before any work starts makes the matrix a by product of working normally rather than a document written afterwards.
D009	Compensating action instead of a multi document transaction	Wrap both writes in a transaction	Transactions need a replica set, which a single instance is not. The failure window is small and the compensation is exact.

Changes made after the artefacts were first agreed

ID	What changed	Why	Artefacts updated
C001	Forms no longer let the browser block submission	The native validation attributes meant the browser showed its own generic wording instead of the messages specified in section 1.9, so the rules written in the requirements were not what the user actually saw	Frontend forms, screenshots, pull request 1
C002	The low seat warning is suppressed while an error is shown	The same sentence appeared twice in two different colors after a rejected booking	Event detail screen, screenshots
C003	Environment file path resolved from the source file	The application crashed on every boot after deployment because the process manager starts from a different working directory	Server entry point, runbook

2. System Design

2.1 Why SysML rather than plain UML

The brief specifies SysML notation. Two of the diagram types below are ones UML does not provide, and they carry the most weight here.

- The requirement diagram makes the satisfy and derive relationships explicit, so the link from a requirement to the block that implements it is part of the model rather than prose.
- Block definition and internal block diagrams describe the system as blocks with ports and interfaces, which suits a client, server and database system better than a class diagram that would only describe one of those three tiers.

The behavioral views, meaning sequence, activity and state machine, are shared between UML and SysML and are drawn here with SysML stereotypes where relevant.

Where the diagrams come from

Figures 2 to 10 are the pages of the draw.io file linked on the cover page, captured from the viewer. They are not redrawn for this document, so what the marker opens through that link is exactly what is printed here. Figure 1 is the one exception. It is a context view written in Mermaid, which renders inline in the repository and has no equivalent page in the draw.io file.

Opening the file in the viewer was worth doing rather than assuming the generated XML was correct. It showed that every message on the sequence page had been drawn on the same horizontal line, overlapping into something unreadable, because the edges were attached to the lifelines and to absolute coordinates at the same time. Draw.io ignores the coordinates in that case. The generator was corrected and the page in figure 6 is the result.

2.2 Context

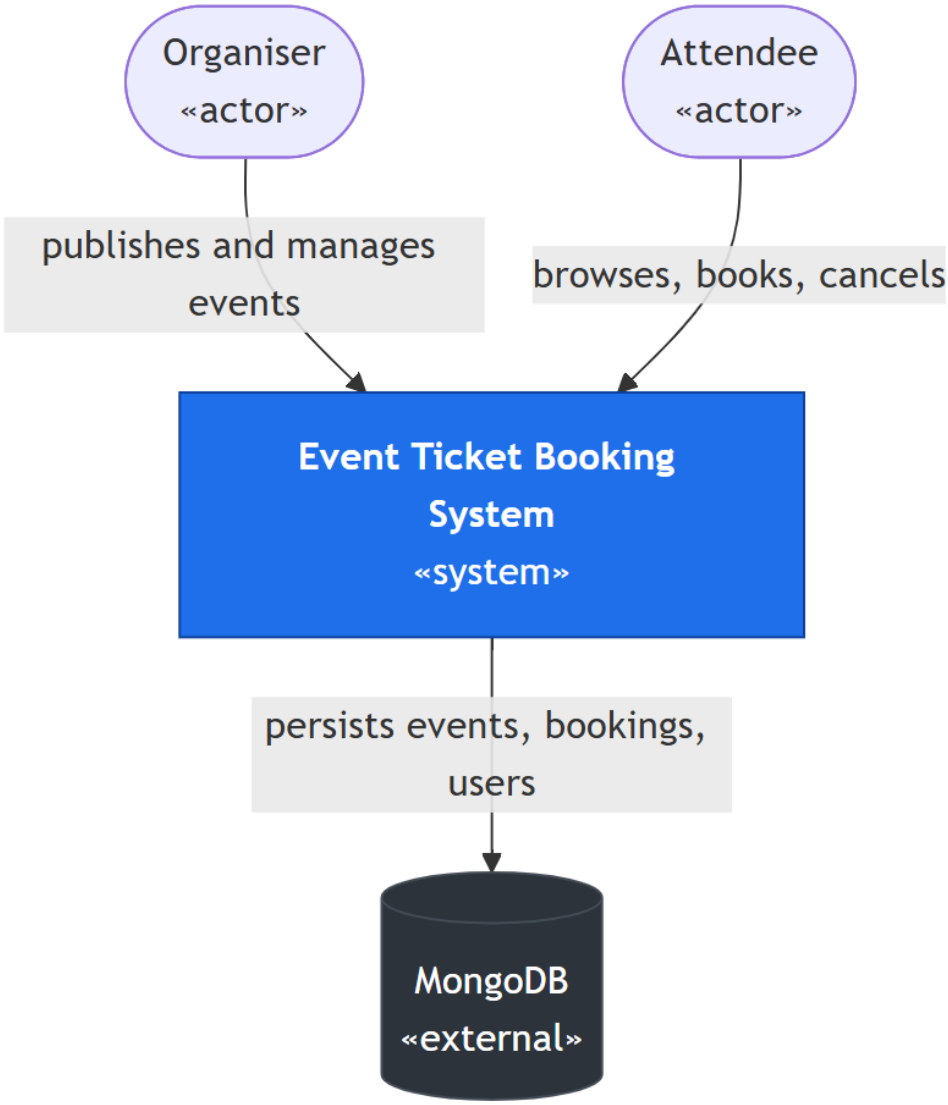

Figure 1. System context. Payment and notification providers are outside the boundary because both are out of scope, and drawing them would misrepresent what was built.

2.3 Use case diagram

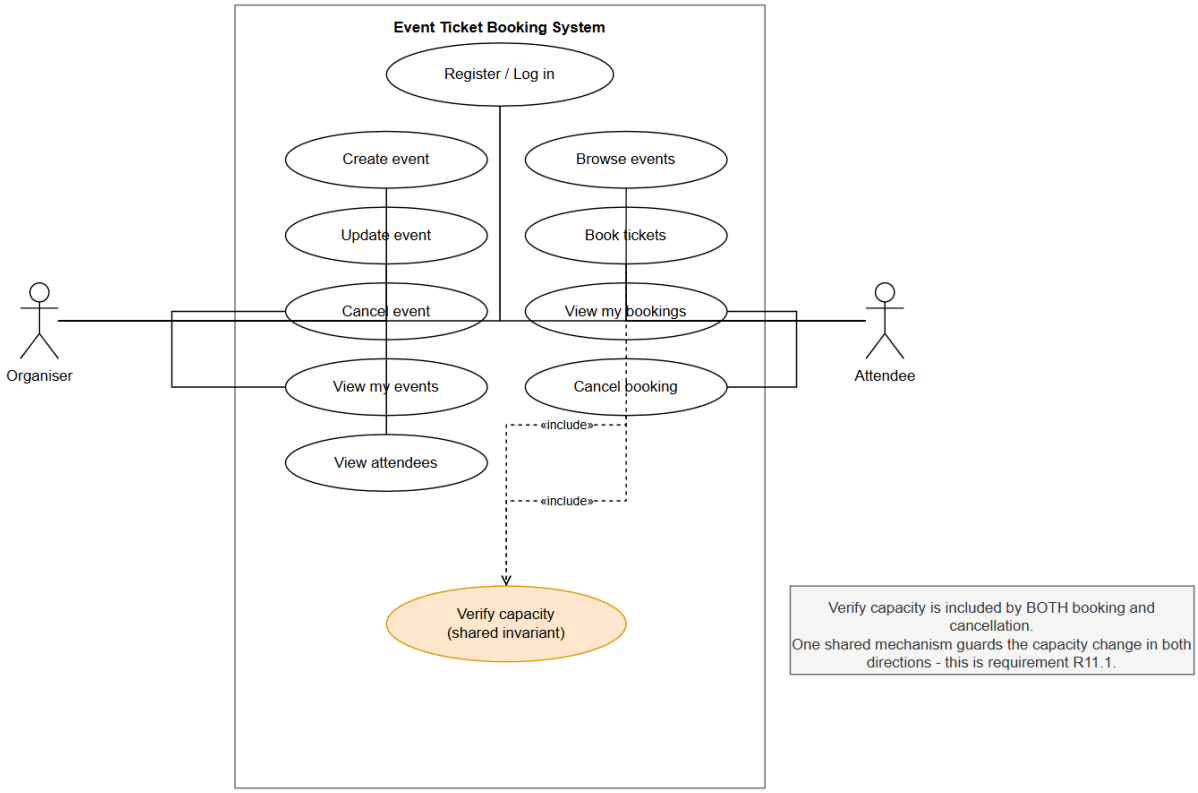

Figure 2. Use cases for both roles. Verify capacity is an included use case on booking and on cancellation, which is the same idea as the derived requirement R11.1: one shared mechanism guards the capacity change in both directions.

2.4 Requirement diagram

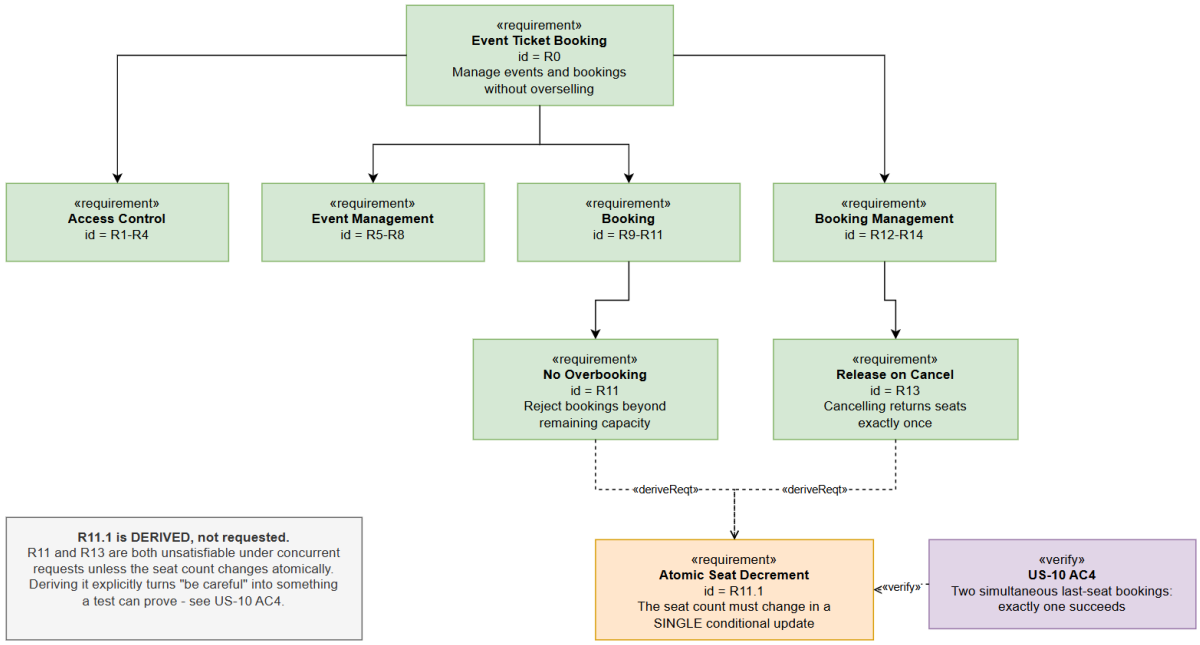

Figure 3. SysML requirement diagram showing how the register decomposes and where R11.1 is derived from.

R11.1 is a derived requirement, not one a stakeholder asked for.

It exists because both R11, do not oversell, and R13, release seats exactly once, are impossible to satisfy under concurrent requests unless the seat count changes atomically. Deriving it explicitly is what turns the vague instruction be careful into something that can be tested, which is acceptance criterion AC4 of US10.

2.5 Block definition diagram

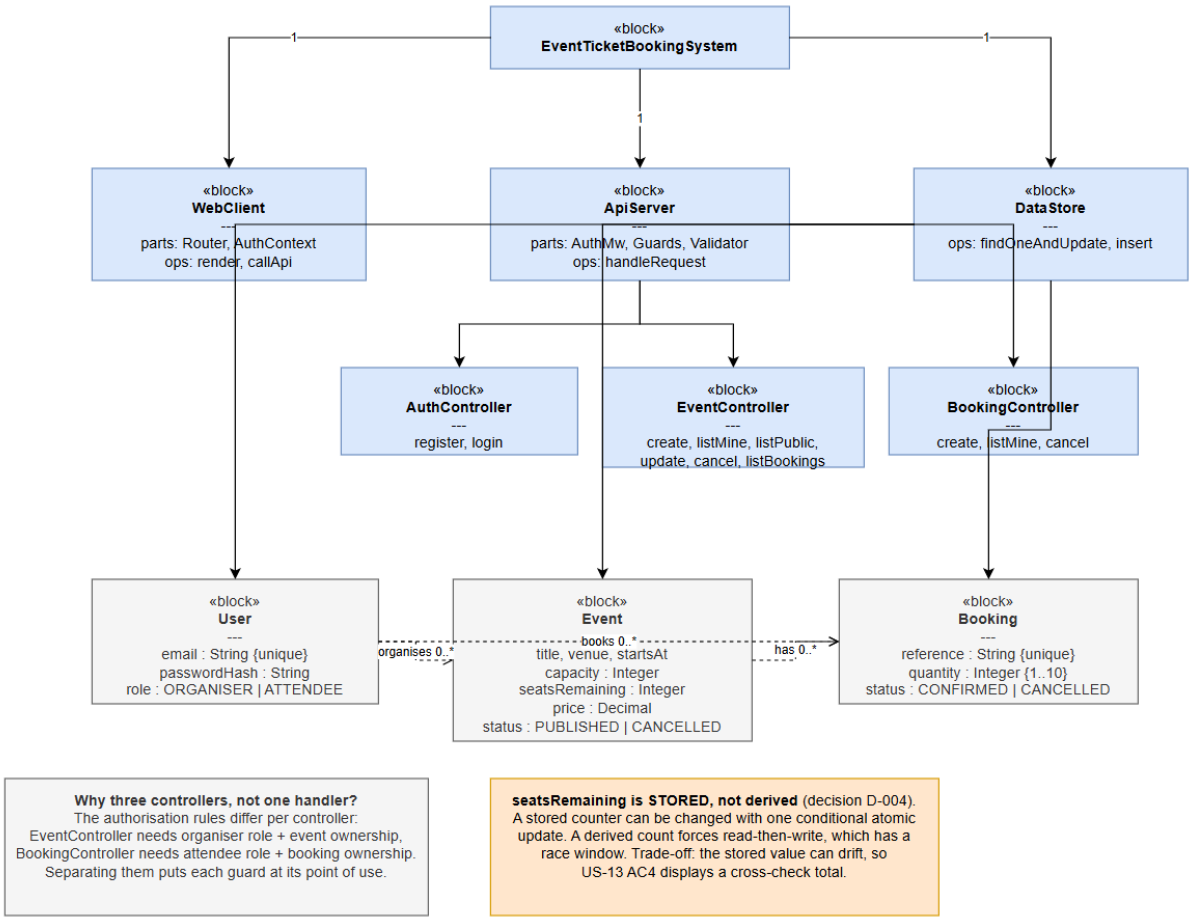

Figure 4. Structural decomposition of the system into blocks.

Controllers are separate blocks rather than one handler because the authorization rules differ per controller. The event controller needs an organizer role plus event ownership, the booking controller needs an attendee role plus booking ownership. Keeping them apart makes each guard obvious at its point of use.

2.6 Internal block diagram

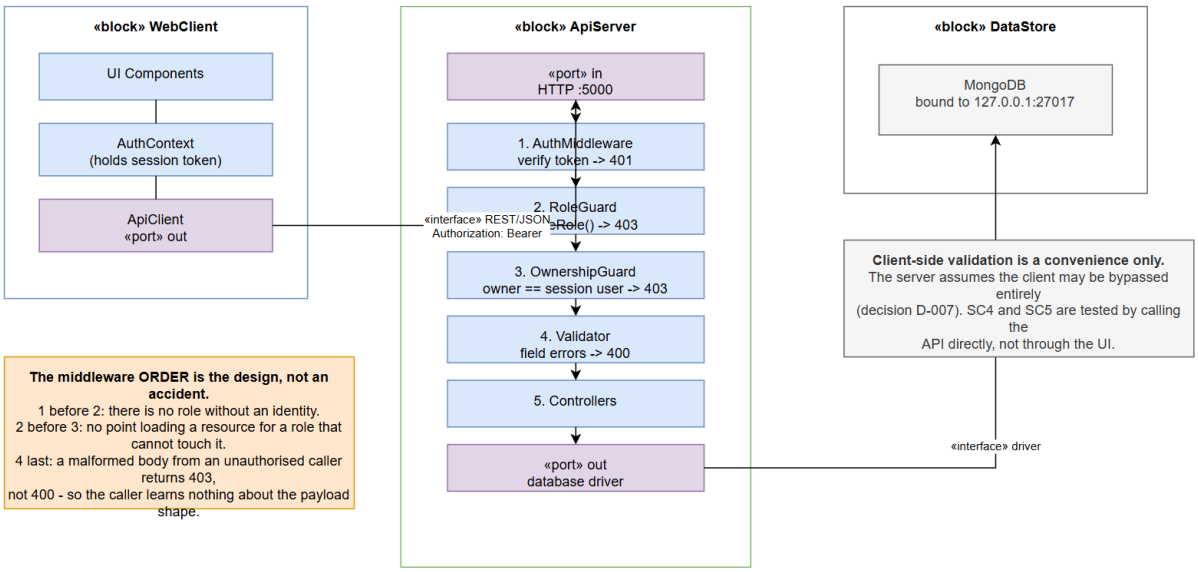

Figure 5. How the blocks connect and across which interfaces.

The middleware order is the design, not an implementation detail.

Authentication must precede the role guard, because there is no role until the identity is known. The role guard must precede the ownership guard, because there is no point loading a record for a role that cannot touch it. Validation runs last, so a malformed body from an unauthorized caller is rejected as forbidden rather than as a bad request, and the caller learns nothing about the expected payload shape.

2.7 Behavioral view for W1

This is the required behavioral view for the deployed workflow. It shows the success path and both rejection paths, because the rejection paths are where the design does its real work.

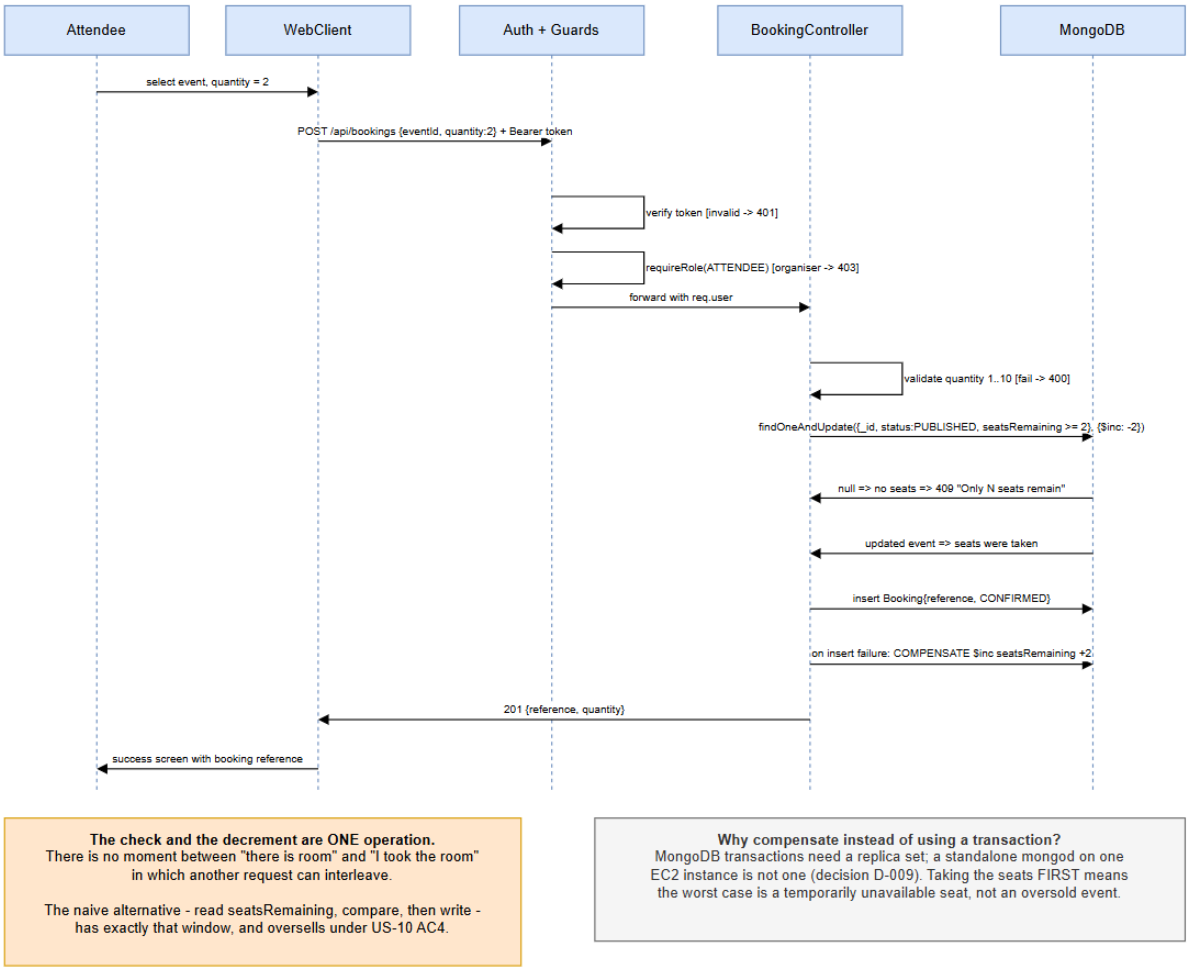

Figure 6. Sequence diagram for W1, booking tickets, including the unauthorized, forbidden, invalid and capacity conflict paths.

Two things in this diagram are the entire point of the design.

Step ordering.

The condition that at least two seats remain sits inside the same database operation as the subtraction, in the filter rather than in a preceding if statement. There is no moment between deciding there is room and taking that room in which another request can interleave.

```
findOneAndUpdate(
  { _id: eventId, status: "PUBLISHED", seatsRemaining: { $gte: quantity } },
  { $inc: { seatsRemaining: -quantity } },
  { new: true }
)
```

The naive alternative reads the count, compares it, then writes the new value back. That has a window between the read and the write, and under two simultaneous requests for the last seat it oversells the event. Whichever request arrives second here matches no document and receives null, which becomes a 409 response.

Compensation.

If the booking record fails to save after the seats were taken, the seats are added back. Without that step a failed save would silently shrink the event capacity forever. This is a compensating action rather than a transaction, which is a deliberate trade off recorded as D009 and explained in section 2.8.

2.8 Behavioral view for W2

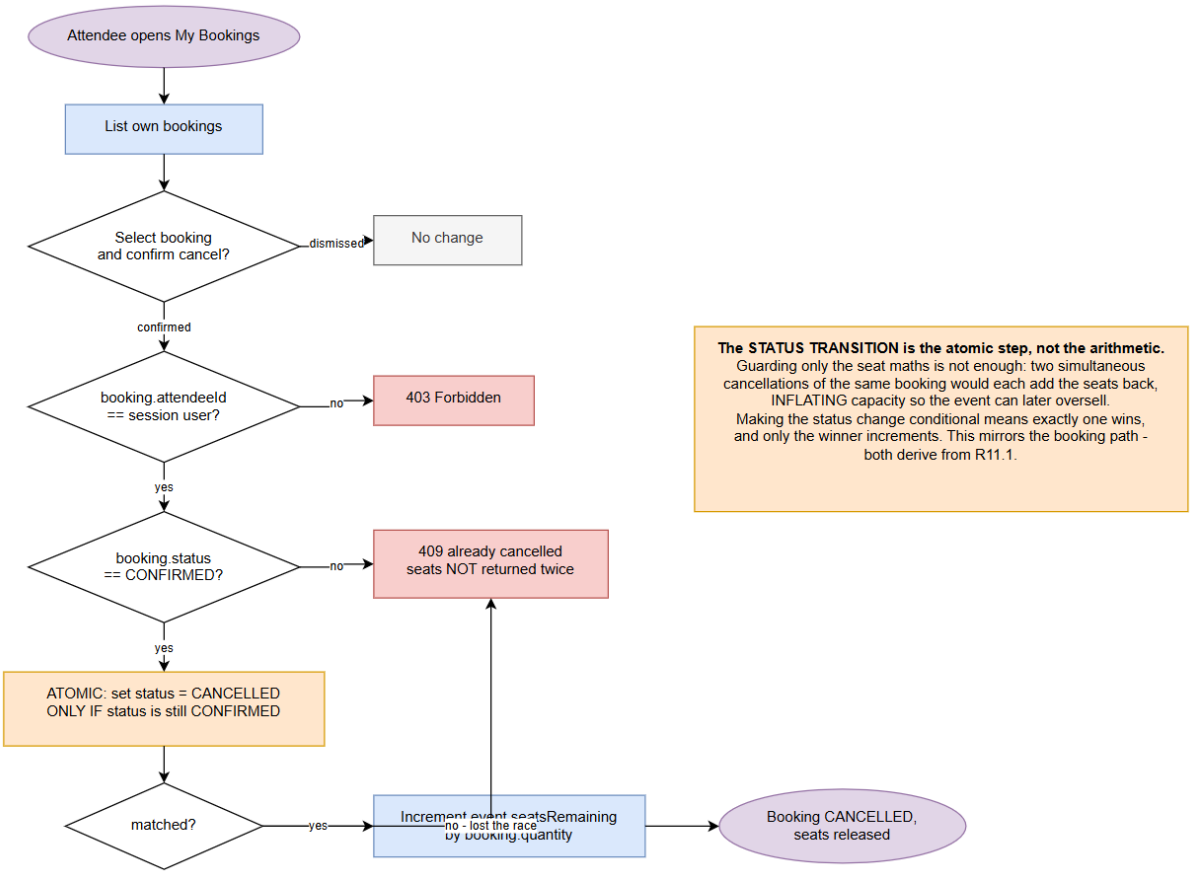

Figure 7. Activity diagram for W2, cancel and release.

The status guard is what prevents capacity inflation.

Guarding only the seat arithmetic is not enough. Two simultaneous cancellations of the same booking would each add the seats back, and the event would then believe it has more room than the venue holds. Making the status transition itself the atomic conditional step means exactly one request wins, and only the winner is allowed to return the seats. This mirrors the booking path, which is why both derive from R11.1.

2.9 State machines and data model

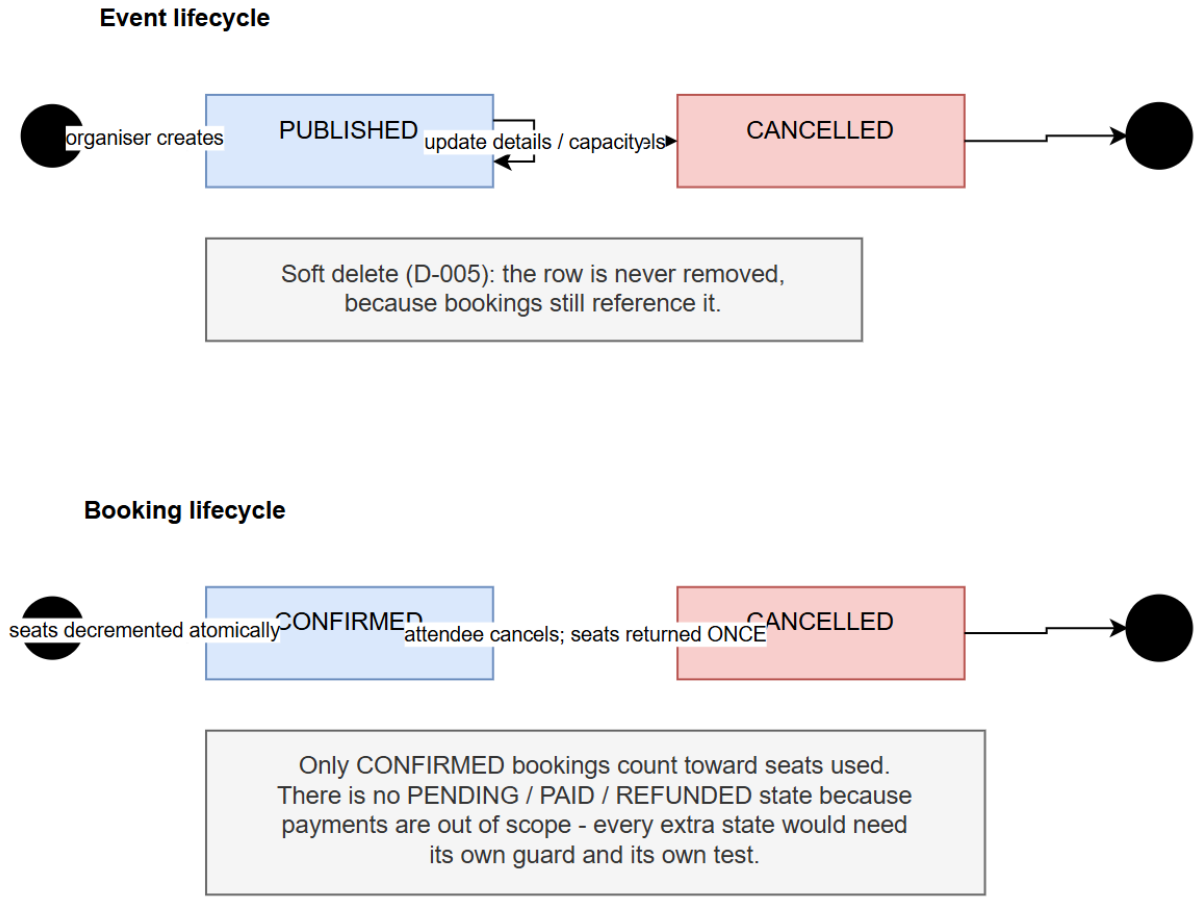

Figure 8. Event and booking lifecycles. Cancellation is a soft delete in both cases, so no record is removed and no booking is ever orphaned.

Both state machines are deliberately small. A booking has two states rather than five because every extra state needs its own guard and its own test, and none of the obvious extras, pending, paid or refunded, are reachable while payments are out of scope.

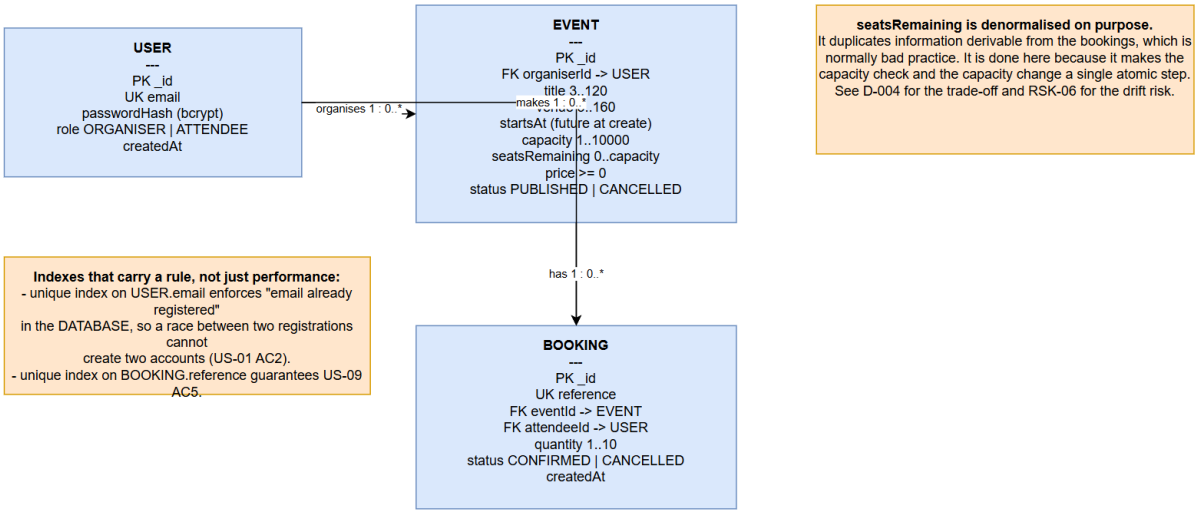

Figure 9. Data model. The note explains why the remaining seat count is stored rather than derived.

The trade off behind D004

Option	Advantage	Disadvantage
Compute the remaining count by summing bookings on every read	Always correct by construction, cannot drift	Enforcing the limit needs a read, a check and a write, with a race window in between
Store the count and change it atomically (chosen)	Can be changed with one conditional update, safe under concurrency without locks	Could drift from reality if a bug updates one without the other

Correctness under concurrency was judged more important than immunity from drift. The drift risk is mitigated rather than eliminated. The attendee list screen shows a confirmed seat total calculated by summing bookings next to capacity minus the stored counter. Those two numbers are produced in completely different ways and must always agree, so any drift becomes visible immediately. Figure 19 shows both numbers agreeing on the deployed system.

2.10 API surface and traceability matrix

Method	Path	Auth	Role	Ownership	Req
POST	/api/auth/register	no			R1
POST	/api/auth/login	no			R2
GET	/api/events	no			R9
GET	/api/events/:id	no			R9
POST	/api/events	yes	organizer		R5
GET	/api/events/mine	yes	organizer	own only	R6
PATCH	/api/events/:id	yes	organizer	must own	R7
POST	/api/events/:id/cancel	yes	organizer	must own	R8
GET	/api/events/:id/bookings	yes	organizer	must own	R14
POST	/api/bookings	yes	attendee		R10, R11
GET	/api/bookings/mine	yes	attendee	own only	R12
POST	/api/bookings/:id/cancel	yes	attendee	must own	R13

Twelve endpoints, each with an explicit authentication, role and ownership rule. The table also serves as the test matrix for SC4 and SC5. Every row with a role gets a wrong role test and every row with ownership gets a wrong owner test.

Traceability matrix

Every cell below points at something that exists and can be checked. Commit hashes are real and are in the repository history. Prototype frame names are generated with the requirement ID as a prefix so the link is unambiguous.

Req	Story and Jira	Design element	Prototype frame	Commit	Deployed evidence
R1	US01, SCRUM 6	AuthController.register	R1 Register	6582394, d397361	Both roles registered, figure 15
R2	US02, SCRUM 7	AuthController.login	R2 Login	6582394, e55dcde	Token returned, figure 16
R3	US02, SCRUM 7	Auth middleware		e29920b	401 with no token
R4	US03, SCRUM 8	Role and ownership guards		e29920b, d923d8b	403 for wrong role
R5	US04, SCRUM 9	EventController.create	R5 Create Event	354b7f0, 93ae389	Event created, figure 18
R6	US05, SCRUM 10	EventController.listMine	R6 Organiser Dashboard	354b7f0, 93ae389	Own events only, figure 17
R7	US06, SCRUM 11	EventController.update	R7 Edit Event	354b7f0, 93ae389	Capacity delta rule
R8	US07, SCRUM 12	EventController.cancel	R8 Cancel Event Confirm	354b7f0, 93ae389	Status set to cancelled
R9	US08, SCRUM 13	EventController.listPublic	R9 Event Listing	354b7f0, 89c62b1	Listing, figure 14
R10	US09, SCRUM 14	BookingController.create	R10 Book Tickets	ddb3691, 89c62b1	Reference ETB CN54 P657
R11	US10, SCRUM 15	Atomic conditional update	R11 Sold Out	ddb3691, 89c62b1	409 returned, figure 20
R12	US11, SCRUM 16	BookingController.listMine	R12 My Bookings	ddb3691, 93ae389	Scoped to caller, figure 21
R13	US12, SCRUM 17	BookingController.cancel	R13 Cancel Booking	ddb3691, 93ae389	Seats returned once, figure 22
R14	US13, SCRUM 18	EventController.listBookings	R14 Attendee List	5539023, 93ae389	Cross check agrees, figure 19
N1	US01, US04	Validator module	Error state frames	6582394, 686e67a	Figures 15 and 18
N2	US01, SCRUM 6	bcrypt hashing		6582394	Hash stored, not plain text
N3	US14, SCRUM 19	gitignore, env example		5ea31db	No secret in history
N4	US15, SCRUM 20	Data store block		57ecd10	Survived restart
N5	US15, SCRUM 20	Deployment view		57ecd10, 82827ad	http://32.236.117.199 returns 200
N6	US15, SCRUM 20	Security group config		57ecd10	Only 80 and 22 open, each to one address

2.11 Deployment view

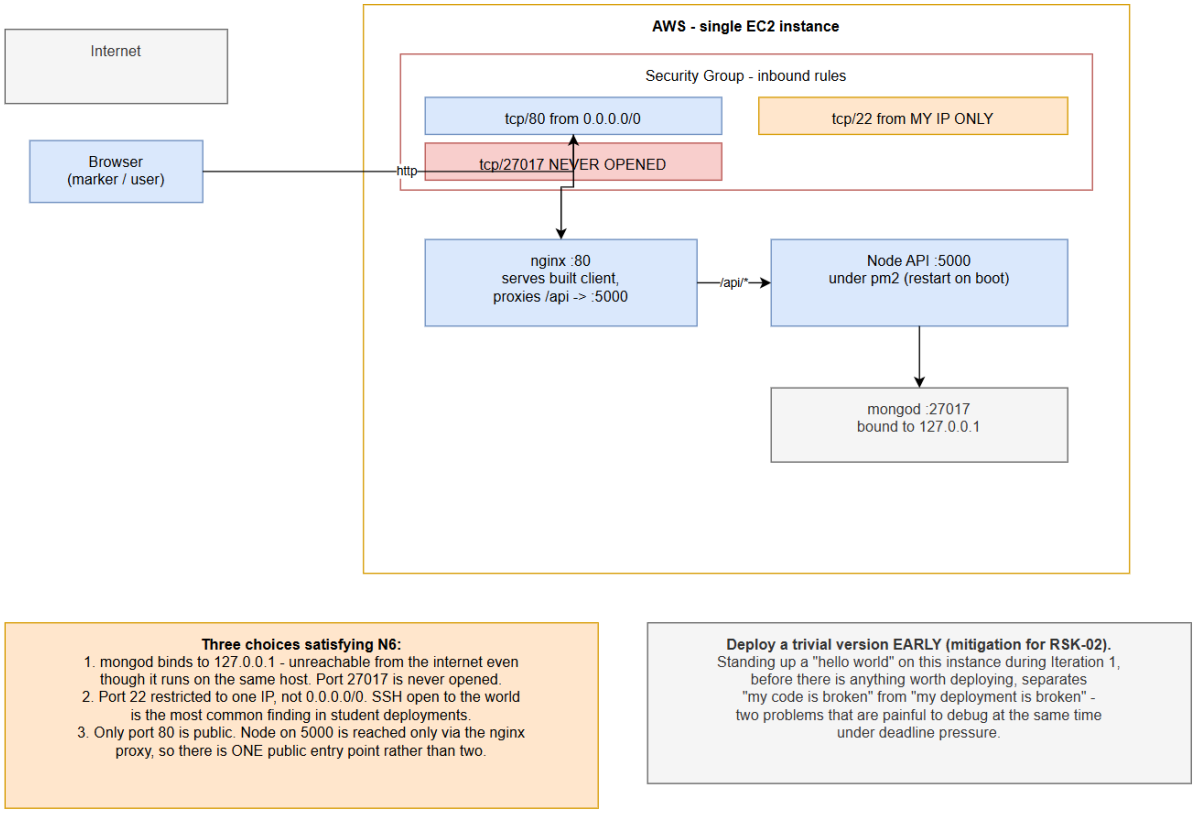

Figure 10. Deployment view. One instance runs the proxy, the API and the database, with only port 80 reachable publicly.

3. User Interface and Experience Design

3.1 Approach

The design was produced in two stages, as the brief asks. Low fidelity wireframes came first, to settle layout and navigation before any visual styling existed, and the high fidelity screens followed.

The Figma file is generated by a plugin rather than drawn by hand. The plugin source is in the repository under design/figma-plugin. Figma has no interface for creating design content from outside the editor, but its plugin interface can create frames, components, text and prototype links. Generating the file this way has three advantages over drawing twenty five frames manually.

1. Consistency is guaranteed, because every screen is assembled from the same nine components and the same spacing constants, so the visual hierarchy cannot drift between screens.
2. It is reproducible. If a requirement changes, the change is made once and the file is regenerated, which matters for the demonstration where a change has to propagate across artefacts consistently.
3. It is reviewable, because the design is version controlled as source and the history shows how it evolved.

3.2 What the design contains

Page	Contents
01 Design System	Color scale, type scale and nine reusable components: three button variants, two field states, three status badges and an event card
02 Wireframes	Six greyscale wireframes, the design phase artefact
03 High fidelity screens	Nineteen screens covering both roles and all four required states, with eleven prototype links

State coverage

State	Where it appears
Normal	Event listing, organizer dashboard, my bookings, create event
Empty	Event listing empty, organizer dashboard empty, my bookings empty
Validation and error	Register with errors, sign in error, create event with errors, capacity conflict
Success	Booking confirmed with the reference shown

Prototype flows

```
W1 attendee path:
  R2 Login -> R9 Event Listing -> R10 Book Tickets -> R10 Booking Success -> R12 My
Bookings

W1 organizer path:
  R1 Register -> R6 Organiser Dashboard -> R5 Create Event -> back to dashboard

W2 cancel and release:
  R12 My Bookings -> R13 Cancel Booking Confirm
```

3.3 The design file itself

The three captures below are the actual Figma file opened in a browser with no account signed in, which is what the marker sees when they follow the link on the cover page. The sign up strip along the bottom is Figma prompting an anonymous visitor, and it is left in deliberately as evidence that the file really is viewable without an account.

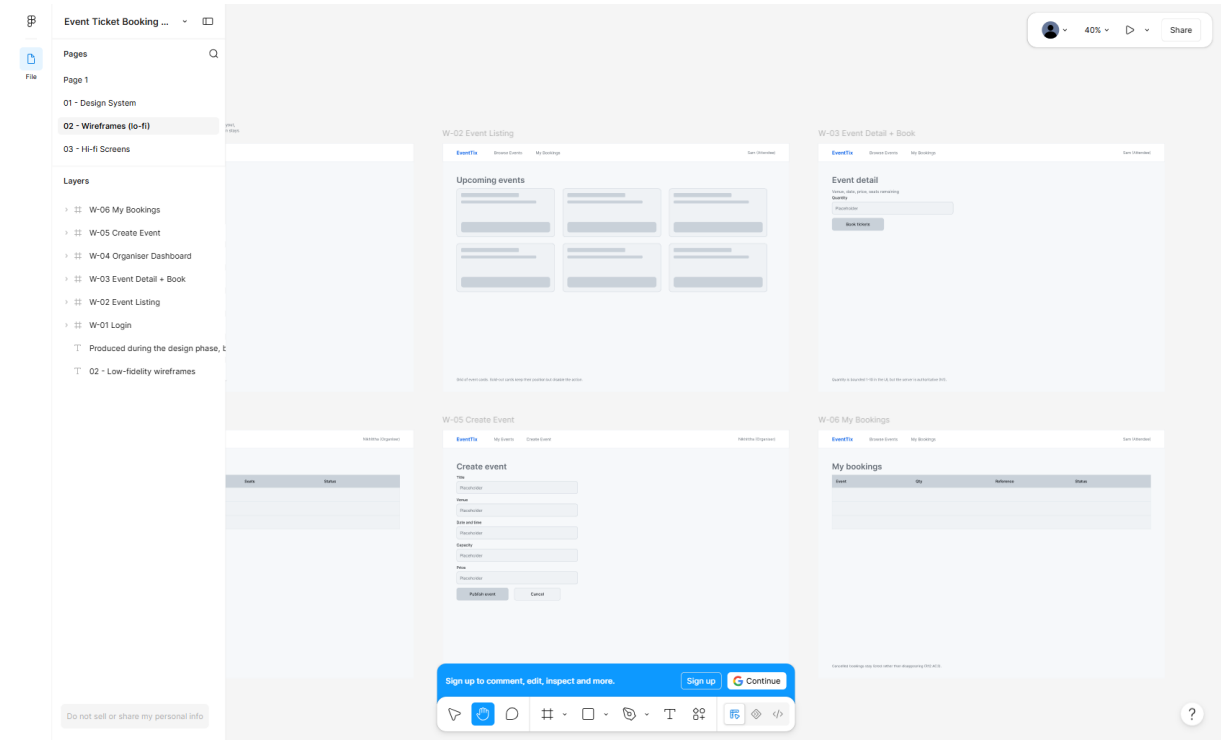

Figure 11. Page 02, the low fidelity wireframes. Six greyscale layouts produced during the design phase, before any visual styling existed, so that discussion stayed on structure and navigation rather than colour.

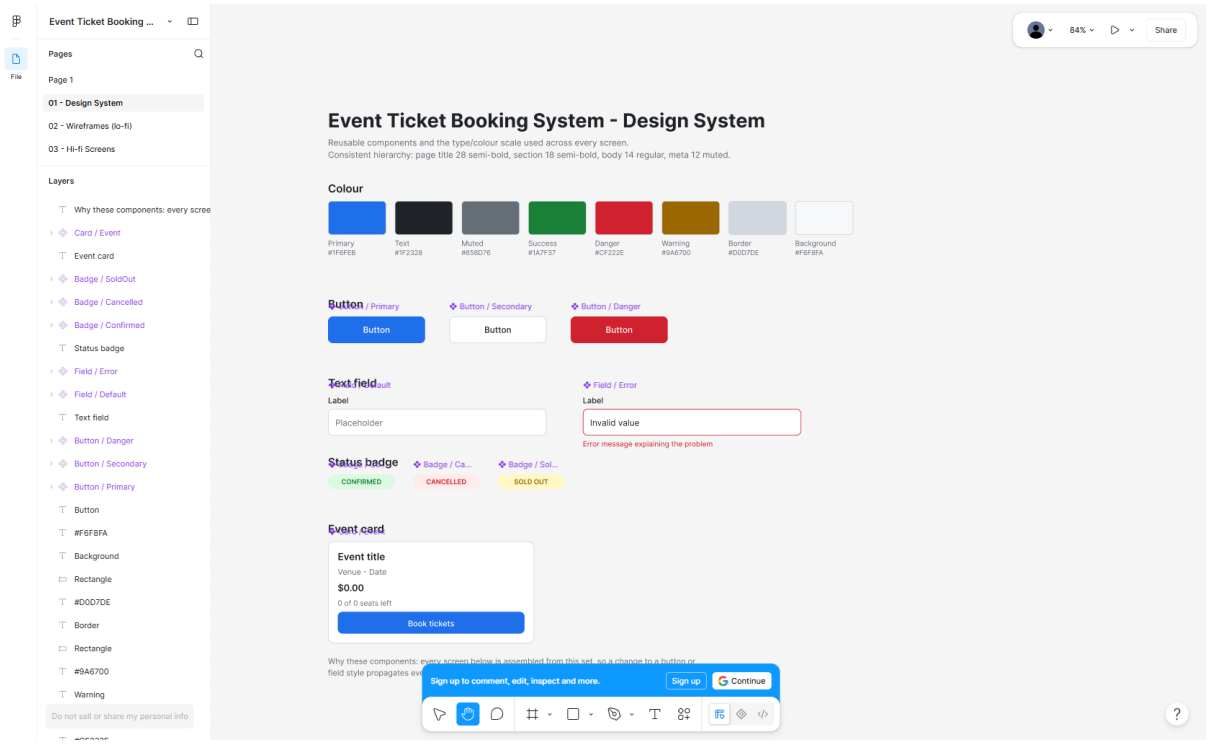

Figure 12. Page 01, the design system. The colour and type scale and the nine reusable components that every screen is assembled from.

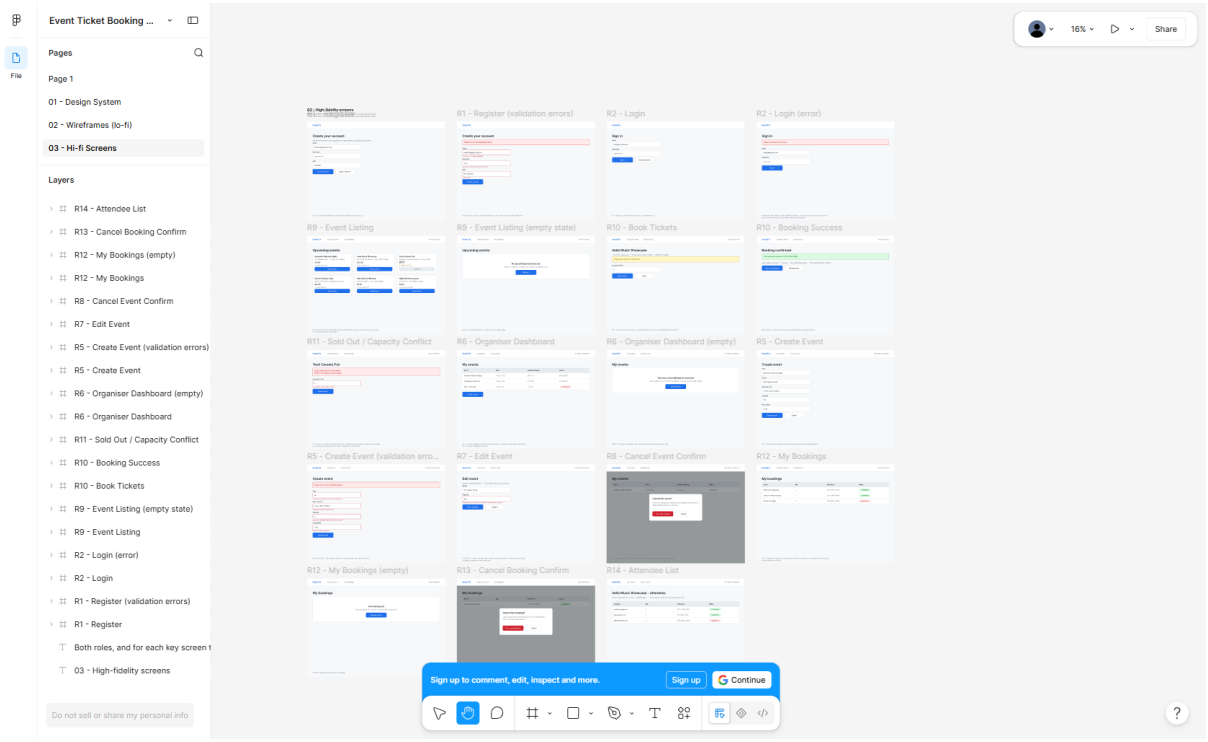

Figure 13. Page 03, the high fidelity screens. Nineteen frames covering both roles and all four required states. The layer names in the panel on the left carry the requirement ID, which is what lets the traceability matrix in section 2.10 point at a named frame.

3.4 Reusable components and hierarchy

Every screen is built from the same nine components, so a change to a button or a field propagates everywhere rather than needing to be repeated. The type hierarchy is fixed at four levels: page title at 28 point semi bold, section heading at 18 point semi bold, body text at 14 point regular and secondary text at 12 point in the muted color. Keeping to four levels is what makes sixteen different screens read as one product.

3.5 The built interface

The screens below are photographs of the running application on the deployed instance, not mockups. They are included here because the brief asks for evidence of a functional prototype rather than pictures of an intended one. Capture is scripted, so the whole set can be regenerated after any change.

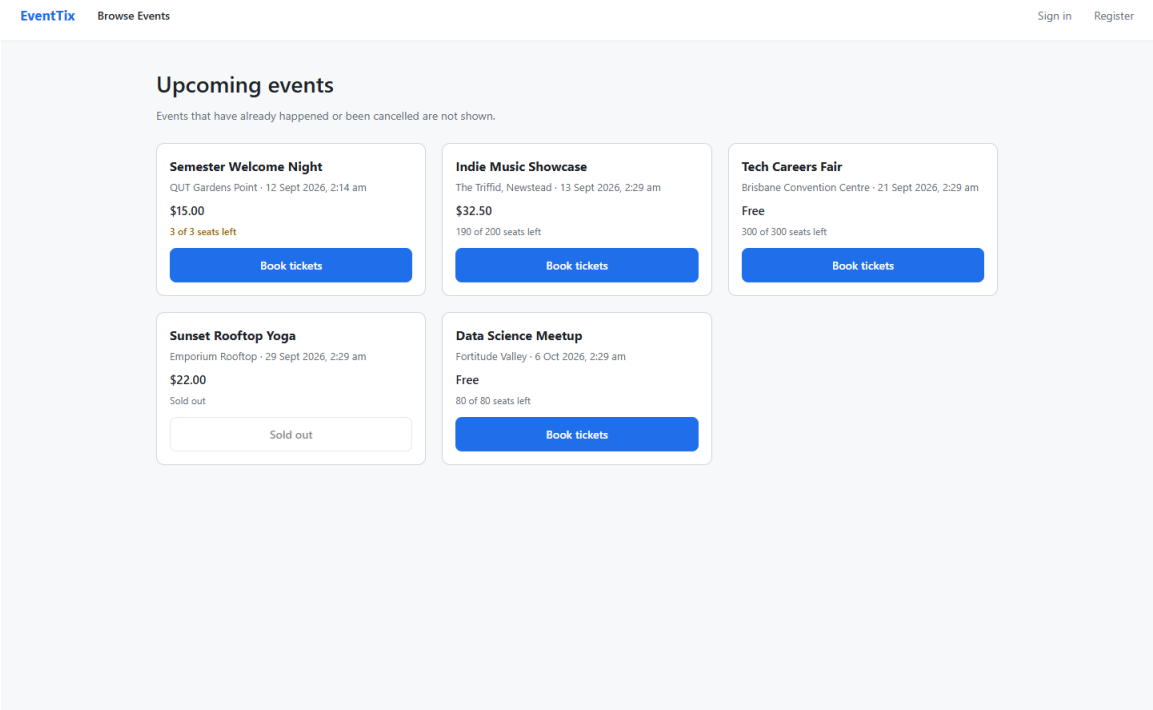

Figure 14. Event listing, normal state. A sold out event keeps its place in the grid but its action is disabled, and events with few seats left are highlighted.

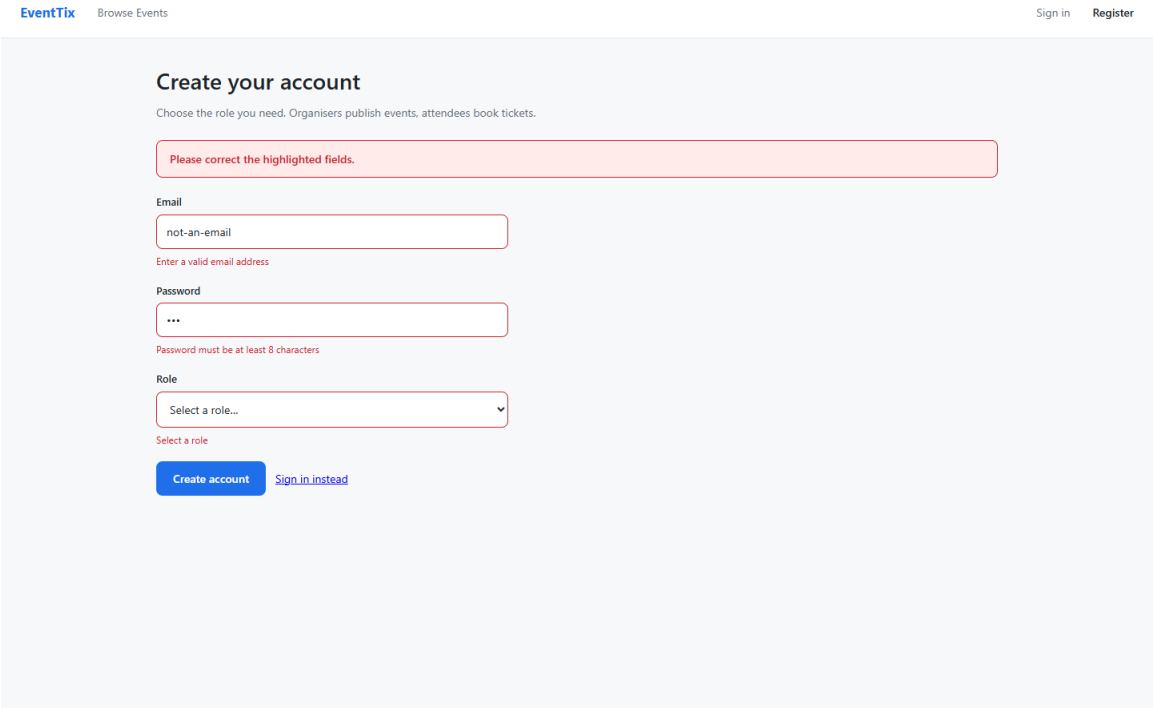

Figure 15. Registration with validation errors. Each message is the exact wording specified in section 1.9.

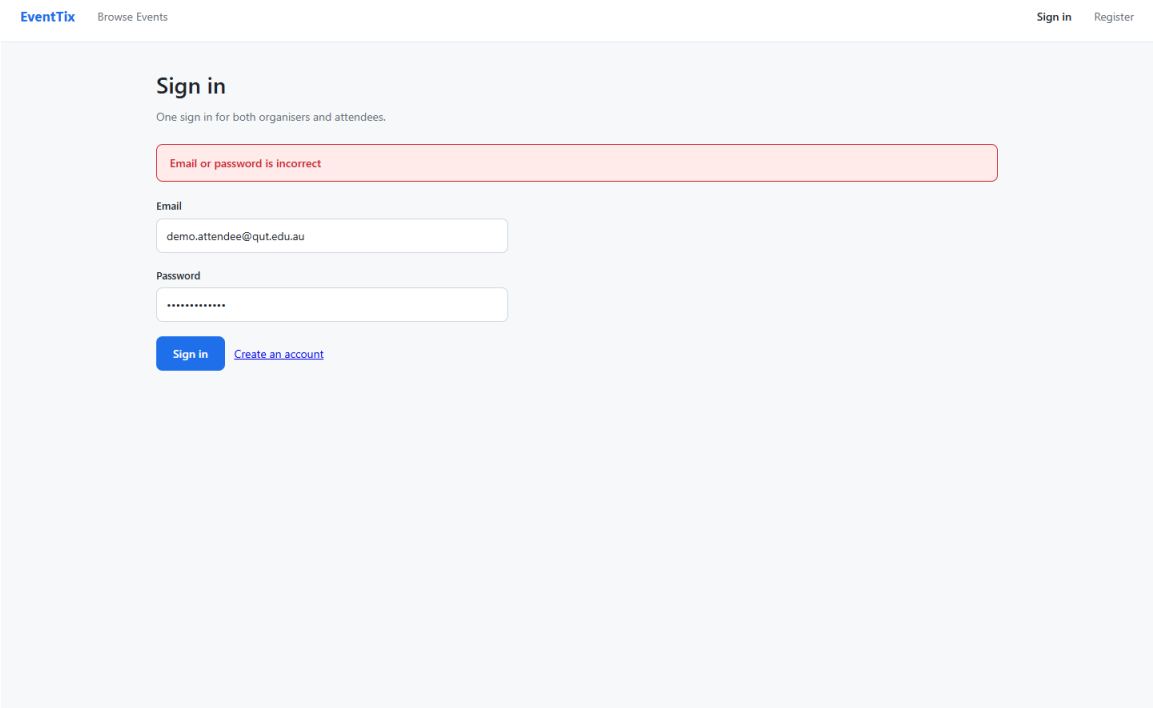

Figure 16. Failed sign in. The message is identical whether the email is unknown or the password is wrong, which is decision D006.

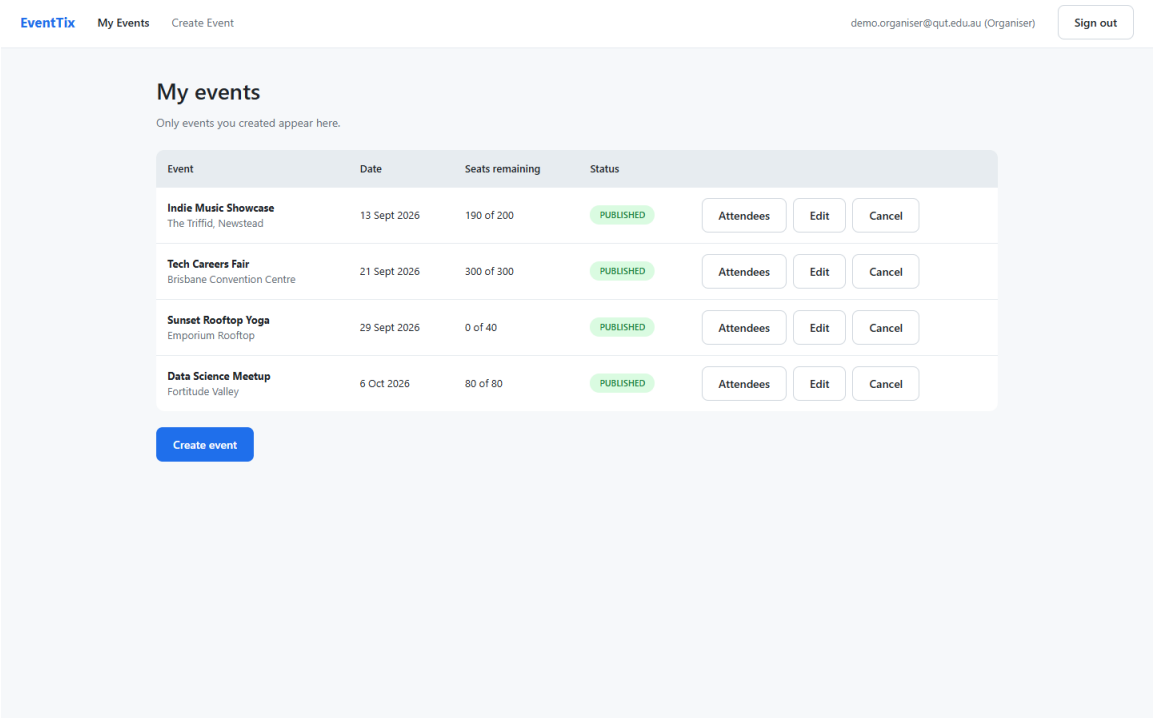

Figure 17. Organizer dashboard, showing only events owned by the signed in organizer.

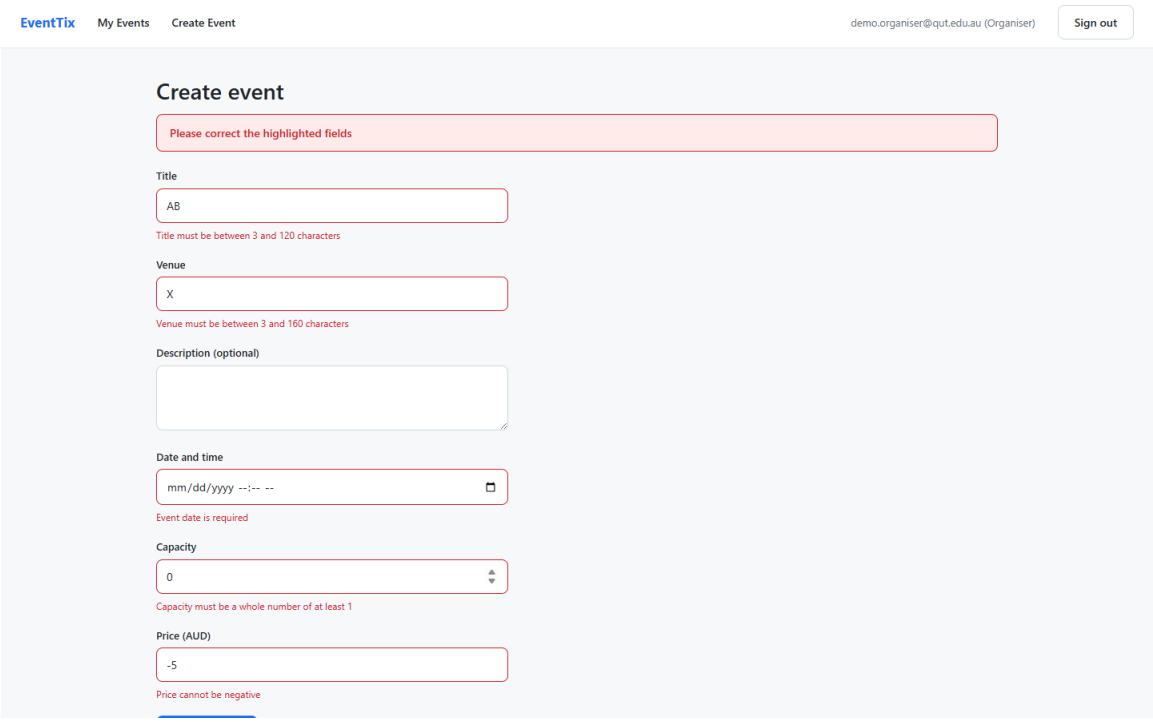

Figure 18. Create event with four validation rules reporting at once.

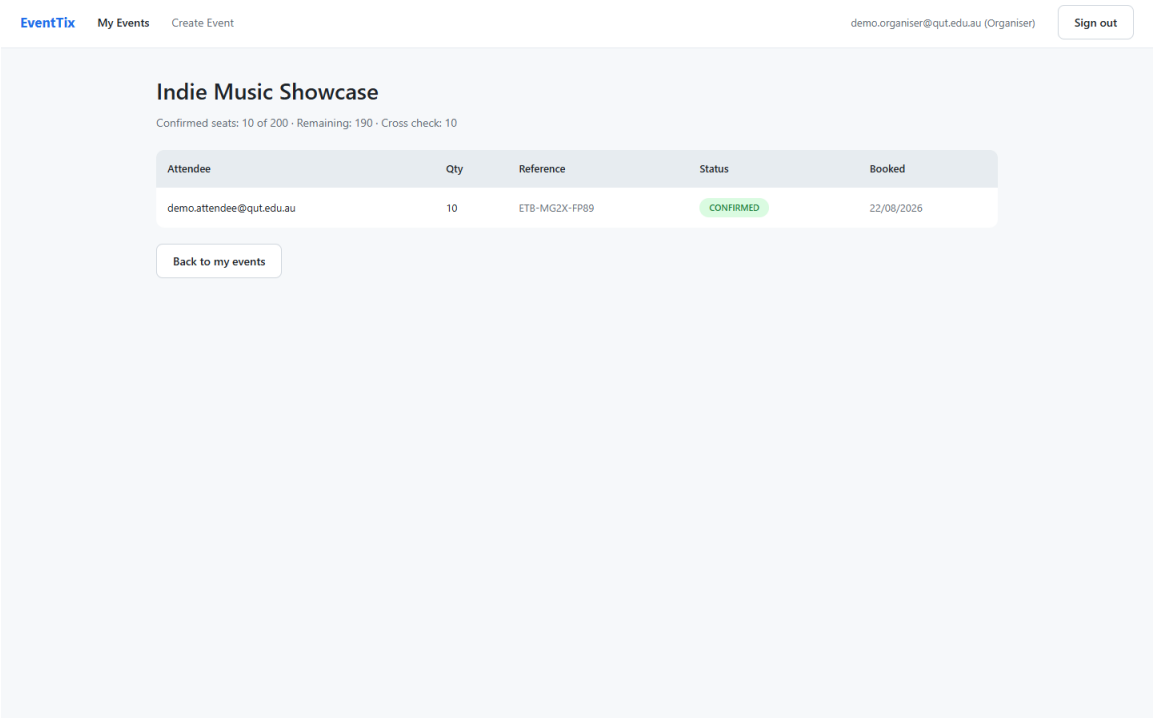

Figure 19. Attendee list. The confirmed seat total and the cross check are calculated independently and agree, which is the safety net for decision D004.

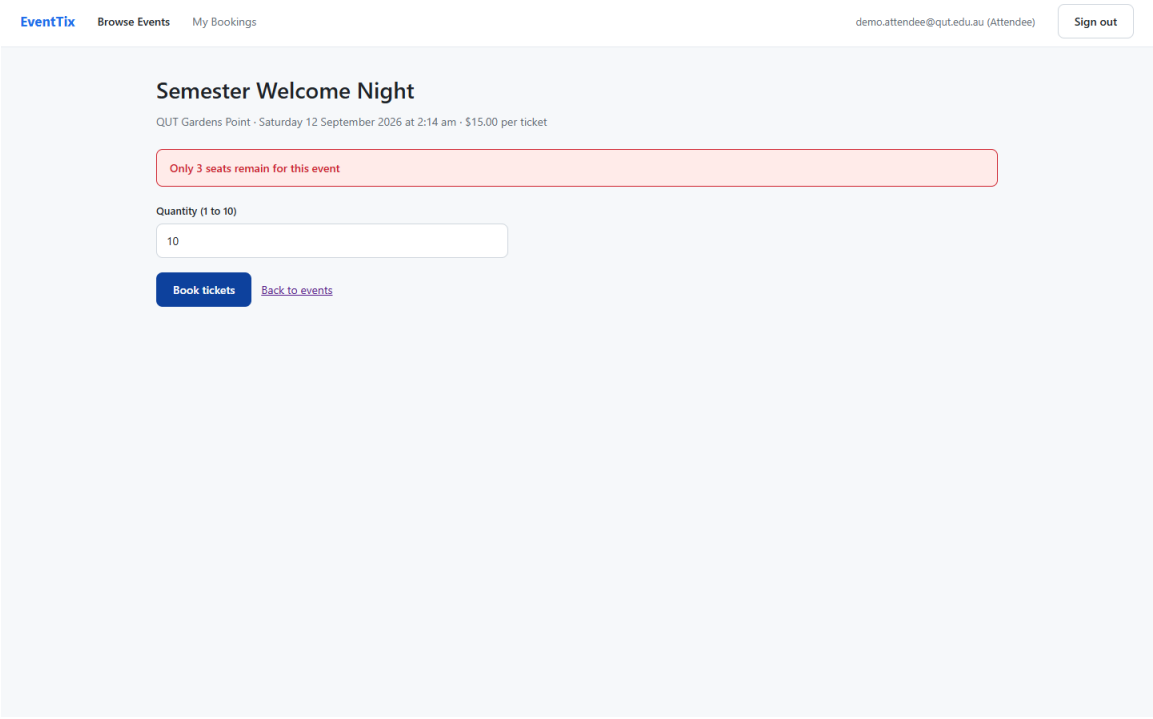

Figure 20. Capacity conflict. The attendee asked for more seats than remain and the request was refused. No booking record was created.

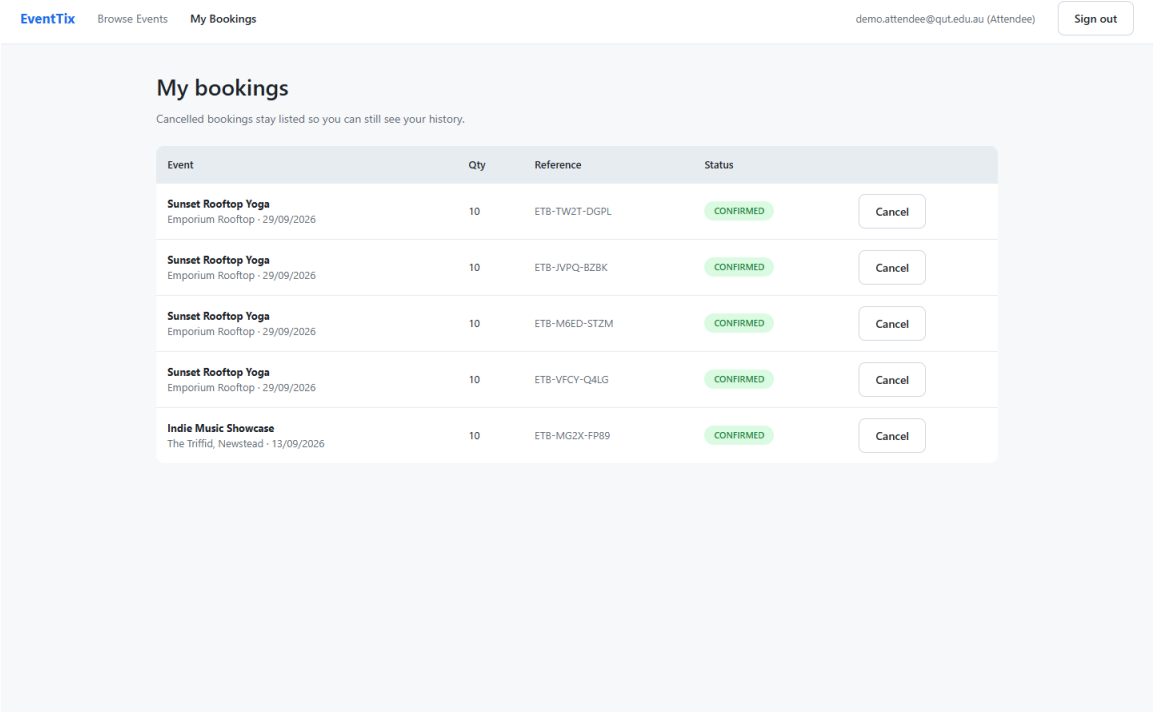

Figure 21. My bookings. Cancelled bookings stay in the list rather than disappearing, which keeps the seat arithmetic auditable.

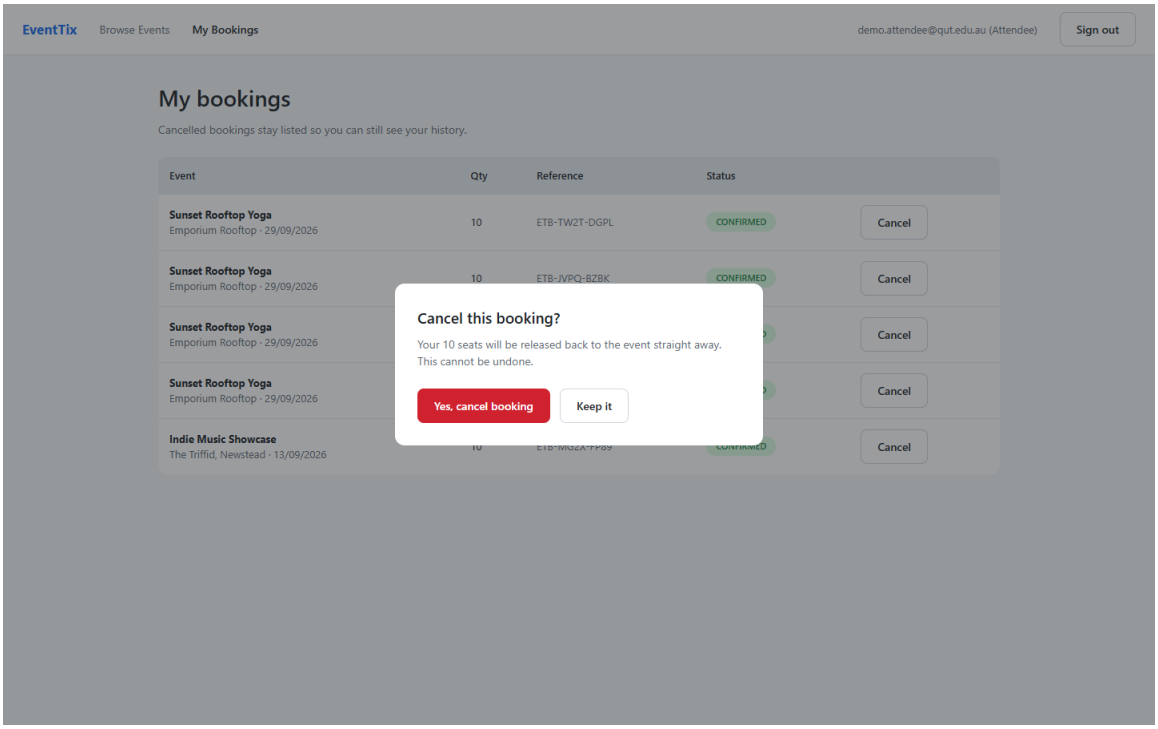

Figure 22. Cancellation confirmation. Nothing changes unless the attendee confirms.

3.6 The generated prototype file

The plugin was run in the Figma desktop application and produced the file linked on the cover page. Local plugin development is only supported in the desktop application and not in the browser, which is why that

step is done there. The instructions are in design/README.md in the repository, so the file can be regenerated from source at any time.

Before submission the file sharing setting must be set to anyone with the link can view, otherwise the marker will be asked to request access. This is a setting on the file rather than anything in the design itself.

4. Git Version Control Practice

4.1 Repository

Item	Value
Repository	https://github.com/nikki505/event-ticket-booking
Visibility	Public, so the marker needs no access request
Account display name	Nikhittha Mukkala
Default branch	main
Commits	25 at the time of writing
Pull requests	1, merged

4.2 Commit practice

Commits were made as the work happened rather than in one batch at the end, and each carries the story and requirement identifiers it implements. That prefix is what makes the traceability matrix in section 2.9 a by product of working normally rather than a document assembled afterwards from memory.

Messages explain reasoning, not just the change. For example the commit that added booking does not say added booking. It records why the capacity condition sits inside the database filter rather than in an if statement, and what would go wrong if it did not.

History was not rewritten and no commit was manufactured after the fact. Where a gap exists it is stated in section 4.5 rather than hidden by editing the past.

4.3 Branching and the pull request

A feature branch, feature/submission report, was used for the final phase of work. It contains three commits and was merged through pull request 1 after a written self review.

What the self review found

Finding 1, fixed. While capturing evidence screenshots, the create event form showed the browser message Value must be greater than or equal to 1 instead of the message specified in section 1.9, Capacity must be a whole number of at least 1. Every form carried native HTML validation attributes, which make the browser block submission and display its own generic wording, so the handler never ran and the server was never asked. This was more than cosmetic. Section 1.9 lists exact wording for every rule and decision D007 states the server is the authority, and both were quietly untrue in the running application. The fix keeps the native attributes, because they still give the correct keyboard and spinner behavior, but stops them blocking submission. Figure 18 shows the result.

Finding 2, fixed. After a rejected booking the page displayed the sentence Only 3 seats remain for this event twice, once as an error and again as a warning directly below it, which reads as though two separate things have gone wrong. The warning is now suppressed while an error is on screen.

Finding 3, recorded rather than fixed. The event form has no client side validation and relies entirely on the server round trip, which is correct and safe but slower than the registration form. This was deliberately not fixed, because adding a second copy of the rules is exactly the duplication that lets the two drift apart. The

right fix is to share one rule module between client and server, and that is recorded as future work in section 6.3 rather than done badly under time pressure.

4.4 Release tag

The submitted state is tagged so the marker can see exactly what was submitted even if work continues afterwards.

```
git tag v1.0 assessment1
git push origin v1.0
```

4.5 An honest gap

The first twenty one commits were made directly on the main branch. Feature branches and pull requests were only introduced for the final phase of work, which is the branch and pull request described in section 4.3.

This is stated plainly rather than concealed. The brief asks for feature branches and pull requests throughout, and for the earlier stories that did not happen. It would have been possible to rewrite history to make it look otherwise, but the brief also forbids rewriting history or manufacturing commits after the work is done, and a fabricated branching history would be a more serious problem than an admitted gap. What the repository shows is what actually happened.

5. Sample Application and EC2 Deployment

5.1 What was built

A working application implementing both workflows, not a mockup. React and Vite on the client, Node and Express on the server, MongoDB for storage, JSON web tokens for sessions and bcrypt for password hashing.

Layer	Technology	Why
Client	React 18, Vite, React Router	Component reuse mirrors the design system, so the built interface matches the prototype
Server	Node 20, Express 4	Middleware chain expresses the authentication, role and ownership order directly
Database	MongoDB 7 with Mongoose	Its single document atomic update is what makes the capacity rule enforceable
Tests	Jest, Supertest, in memory MongoDB	Tests need no running database and cannot touch development data

5.2 Automated tests

Forty nine automated tests across four suites, all passing. The suite is not padded. It concentrates on the cases where the implementation could plausibly be wrong.

Suite	Tests	What it proves
capacity	10	The event cannot be oversold, including under simultaneous requests
permissions	14	Wrong role and wrong owner are refused on every protected route
events	13	Validation rules, the capacity delta rule and soft deletion
cancellation	12	Seats return exactly once, including under simultaneous cancellations

The two tests that matter most

Two tests fire genuinely concurrent requests rather than sequential ones. The first sends two bookings for the last remaining seat at the same moment and asserts that exactly one succeeds and the other receives a conflict response, with the remaining count never going below zero. The second sends ten requests at three available seats and asserts that exactly three succeed.

These are the tests that would fail if the implementation read the seat count and then wrote it back. Every other test in the capacity suite passes with that broken implementation, which is precisely why the concurrent tests were written before the story was called done.

5.3 Deployment and the account restrictions that shaped it

Item	Value
AWS account	438807478932, qut aws learn teach 5
Region	ap-southeast-2, Sydney

Instance ID	i-04a1250e9732b2449
Instance name	n12202665-nikhittha-eventtix
Instance type	t3.micro, Amazon Linux 2023
Subnet	subnet-01b6baa7effb222fc, public
Security group	sg-0cc12e170c4f78181, student-allowed-sg-7
Public URL	http://32.236.117.199
Elastic IP	eipalloc-0538548177cecdd19, so the address survives a stop and start

How the deployment is performed

The instance is launched with a bootstrap script supplied as user data, which the operating system runs once on first boot. Nothing is typed over a remote shell to deploy. This was a deliberate choice. A deployment performed by hand exists only in the memory of the person who did it, whereas a scripted deployment is its own documentation, can be repeated exactly, and can be read by a marker. If the instance were lost, launching a new one with the same script reproduces it.

The script installs the runtime, installs the database and binds it to the loopback address, clones the repository, builds the client, generates the session secret on the instance, starts the API under a process manager configured to restart on boot, and configures the proxy.

Security hygiene

Measure	Detail
Database not reachable from outside	MongoDB binds to 127.0.0.1, so although it runs on the same host as the API it cannot be reached from the internet. Port 27017 is never opened.
Remote shell access restricted	Port 22 is open to a single address, never to the whole internet.
One public entry point	Only port 80 is public. The API on port 5000 is reachable only through the proxy.
Secret generated on the instance	The session secret is created at first boot with a random generator. It is not in the repository, not in the bootstrap script and not passed on a command line.
No secret in version control	The ignore file was the first commit in the repository, before any code existed, so no credential could enter the history.

The restriction that shaped the security group

This is a shared teaching account holding 169 instances belonging to other students. The student role is explicitly denied permission to create a security group in any network in the account, so a purpose built group for this project was impossible. The only option is one of eight pre existing shared groups, each already attached to between 30 and 134 other students instances.

The least used group with no existing rules on port 80 was chosen, so that adding a rule disturbs the fewest people. Because the group is shared, any rule added applies to every instance using it. Port 80 was therefore opened to a single address rather than to the whole internet. Opening it globally would have exposed port 80

on roughly thirty other students instances without their knowledge, which is not a defensible thing to do to shared infrastructure for the sake of one assignment.

5.4 Verification on the deployed instance

Both workflows were walked against the public address, not only in local tests.

Step	Action	Result
1	Register an organizer and an attendee	Both accounts created
2	Publish an event with capacity 3	Appears in the public listing
3	Book 2 tickets	Reference ETB CN54 P657, one seat remaining
4	Organizer views the attendee list	Confirmed 2, cross check 2, the two agree
5	Attempt to book 4 more with 1 remaining	Refused, Only 1 seat remains for this event
6	Organizer attempts to book	Refused, 403
7	Attendee calls an organizer route	Refused, 403
8	Request a protected route with no token	Refused, 401
9	Cancel the booking	Status cancelled, seats returned to 3
10	Cancel the same booking again	Refused, and the count stays at 3 rather than rising to 5
11	Restart the application process	The event and its bookings are still present
12	Stop the whole instance and start it again	Address unchanged, application back on the first request, all data present

Step 10 is the one worth pausing on. If the cancellation guarded only the seat arithmetic and not the status transition, the second cancellation would have added the seats a second time and the count would read 5 for an event with 3 seats. The event could then oversell later. The result of 3 is the evidence that the guard is on the transition.

Step 12 was not planned. The instance was found stopped two days after deployment, which the teaching account appears to do on a schedule, since all 169 instances in it were stopped when the account was first surveyed. Restarting it turned into a better test than the one that was planned. A full machine stop and start exercises the boot configuration, not just the process manager, and it confirmed three things at once: the Elastic IP held the address, pm2 brought the application back with no manual step, and the events and bookings created before the stop were all still there.

It also produced a practical instruction for the demonstration, which is to check the instance is running before the session rather than assume it. That is now step one of the routine operations in the runbook.

5.5 Known limitations

Stated honestly, because a claim that a system has no weaknesses is never true and is easy to disprove.

- The connection is plain HTTP with no transport encryption, because a certificate needs a domain name and there is no domain for this project. Tokens and passwords therefore cross the network in the clear. In a real system this would be the first thing fixed.
- The deployed address is reachable from one address only, for the shared security group reason described in section 5.3.

- An Elastic IP is attached, so the address is stable across a stop and start. It is a shared account resource and should be released once marking is finished.
- A single instance with no load balancer, so it is a single point of failure and there is no zero downtime deployment.
- The database has no authentication enabled. It is bound to the loopback address so it cannot be reached from outside, but any process on the instance could connect.
- There are no backups. The storage volume is set to be deleted when the instance is terminated.
- The remaining seat count is a stored number, so it could in principle drift from the real booking total. This is mitigated by the visible cross check but not eliminated.
- Deployment is manual, as the brief specifies.

5.6 Two items needing action before marking

Both are stated here rather than discovered by the marker.

Access to the deployed application.

The public address currently admits one address. The marker will not be able to open it from their own network. The preferred resolution is to ask the tutor for the marking address and add that single entry, which lets the marker in without exposing anyone else. Several addresses already present on these shared groups appear to be campus addresses, which suggests this is the intended pattern.

Access to the Jira board.

The board is inside a QUT tenanted Atlassian site. Whether the marker needs an explicit invitation or already has access through the same tenancy has to be confirmed with the tutor.

6. Generative AI Disclosure and Reflection

6.1 Declaration

Generative AI was used in the preparation of this assessment. This section records where, for what purpose, and what was verified or changed afterwards.

Tool	Model	How it was accessed
Claude	Claude Opus, Anthropic	Interactive session through Claude Code

6.2 Evidence log

Artefact	What AI was used for	What was verified afterwards
Requirements and scope	Drafting the problem statement, stakeholders, scope, assumptions and the requirements register	Each requirement checked to be individually testable and mapped to a workflow
Backlog and iteration plan	Drafting epics, stories, acceptance criteria, estimates and the risk register	Every story checked to reference a requirement ID. Review outcomes left blank rather than invented.
System design	Producing the SysML views, behavioral views and the deployment view	Each diagram checked against the requirements register and the built system
Application code	Implementing the models, controllers, routes, guards and the React screens	49 automated tests written and run. Both workflows walked manually on the deployed instance.
Prototype generator	Writing the Figma plugin that builds the design file	Executed against a stubbed interface before use, then checked visually
Deployment	Writing the bootstrap script and the runbook	Deployed for real. Two defects found and fixed, described in section 6.3.
This document	Drafting and assembling	Every figure, hash, identifier and address checked against the live system

Extent of assistance, stated plainly

This project was AI assisted end to end. The requirements, backlog, design, application code, tests, deployment scripts, this document and the wording of the reflection in section 6.3 were all drafted with AI assistance. It would be misleading to present any large section of it as unaided work, so no such claim is made here.

What was decided by the student: the project domain, the tutorial and tutor details, running the prototype generator, and the choices put to me during the work, including whether to open the shared security group to the whole internet, which was declined for the reason given in section 5.3.

What the reflection in section 6.3 describes really happened. The concurrency problem, the environment file fault after deployment and the validation defect found during the pull request review are all recorded in the

repository history and are verifiable there. The events are real; the prose describing them was drafted with assistance.

Before submitting, read section 6.3 and rewrite it in your own words. A reflection is a personal account and is not worth much second hand, and you should be able to defend every sentence of it in the demonstration.

6.3 Individual reflection

The challenge

The requirement that looked simplest turned out to be the one that needed the most thought. R11 says the system must reject any booking that exceeds remaining capacity. My first instinct was to read the event, compare the remaining seats with the requested quantity, subtract if there was room, and save. That is the obvious shape and it is what I would have written without thinking further.

It is also wrong, and wrong in a way that is almost invisible. Every test I could perform by hand passed. Booking more seats than remained was refused. Booking exactly the remaining number worked. Booking against a sold out event was refused. The fault only appears when two requests arrive close enough together that both read the seat count before either writes it back. Both then believe there is room, both save, and the event is oversold. The stored count can even go negative.

The response

Rather than treating this as an implementation detail, I made it a requirement in its own right. R11.1, shown in figure 3, states that the seat count must change in a single conditional operation, and it is derived from both R11 and R13 because cancellation has the mirror problem. Two simultaneous cancellations of one booking would return the seats twice and inflate capacity above what the venue holds.

Naming it as a derived requirement changed how I worked on it. It stopped being the vague instruction be careful about concurrency and became something with an acceptance criterion, AC4 of US10, that either passes or fails. It also explains why US10 is the only story in the backlog estimated at eight points. The estimate reflects risk, not volume of code.

The implementation puts the capacity condition inside the database filter rather than in a preceding conditional statement, so the check and the subtraction cannot be separated by another request. Cancellation applies the same idea to the status transition rather than to the arithmetic, so only the request that wins the transition is permitted to return the seats.

The evidence

I wrote two tests that fire genuinely concurrent requests. One sends two bookings for the last seat and asserts that exactly one succeeds. The other sends ten requests at three seats and asserts that exactly three succeed. Both pass, and both would fail against my first instinct. The behavior was then confirmed on the deployed instance, where cancelling the same booking twice left the count at three rather than raising it to five, as recorded in section 5.4.

What I would do differently

Two things.

First, I would treat deployment as a real risk from the beginning rather than as a final step. The application crashed on every start after its first deployment, claiming a required setting was missing, while the settings file sat in the correct directory. The cause was that the library which reads that file resolves it against the

working directory, and the process manager starts the application from somewhere else. It worked on my machine only because I happened to run it from inside the server folder. Nothing in my testing could have caught that, because the fault was in the difference between the two environments rather than in the code. Deploying something trivial early, before there was anything worth deploying, would have separated infrastructure problems from application problems while there was still time.

Second, I would share one validation module between client and server instead of writing the rules twice. Section 1.9 lists exact wording for every rule and I wrote that wording in two places, which is how the defect in section 4.3 was able to hide. The rules were correct on the server the whole time, but the browser was intercepting the form and showing its own wording instead, so the user never saw the messages the requirements specified. One shared module would have made that impossible rather than merely unlikely.

The wider lesson is the one the unit is about. Both problems were failures of verification rather than failures of coding. In each case the code did what I intended and my testing was not shaped to notice that my intention was incomplete.

6.4 References

Amazon Web Services. (2026). Run commands on your Linux instance at launch. AWS documentation. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html

Amazon Web Services. (2026). Security group rules for different use cases. AWS documentation. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules-reference.html

MongoDB. (2026). Atomicity and transactions. MongoDB manual. https://www.mongodb.com/docs/manual/core/write-operations-atomicity/

MongoDB. (2026). db.collection.findOneAndUpdate. MongoDB manual. https://www.mongodb.com/docs/manual/reference/method/db.collection.findOneAndUpdate/

Object Management Group. (2019). OMG systems modeling language (OMG SysML) version 1.6. https://www.omg.org/spec/SysML/1.6/

OWASP Foundation. (2021). A07:2021 identification and authentication failures. OWASP top ten. https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/